

Ministère de la Transition Numérique

Et de l'Innovation Technologique

Ecole Supérieure Africaine des Technologies

De l'Information et de la Communication



Année universitaire 2025-2026

Institut National des Postes

Et Communications



## **MÉMOIRE DE FIN DE CYCLE POUR L'OBTENTION DU DIPLÔME DE MASTER EN SÉCURITÉ INFORMATIQUE ET TECHNOLOGIES DU WEB (SITW)**

---

**SUJET : CONCEPTION ET MISE EN ŒUVRE D'UN FRAMEWORK  
INTELLIGENT DE DÉTECTION DES MALWARES ET ROOTKITS  
DANS LES ENVIRONNEMENTS CLOUD VIRTUALISÉS.**

---

Présenté par :

**Mlle Moayé Line Esther KOUASSI**

Période de stage :

**05 MARS 2026 – 10 JUILLET 2026**

**Maître de stage:**

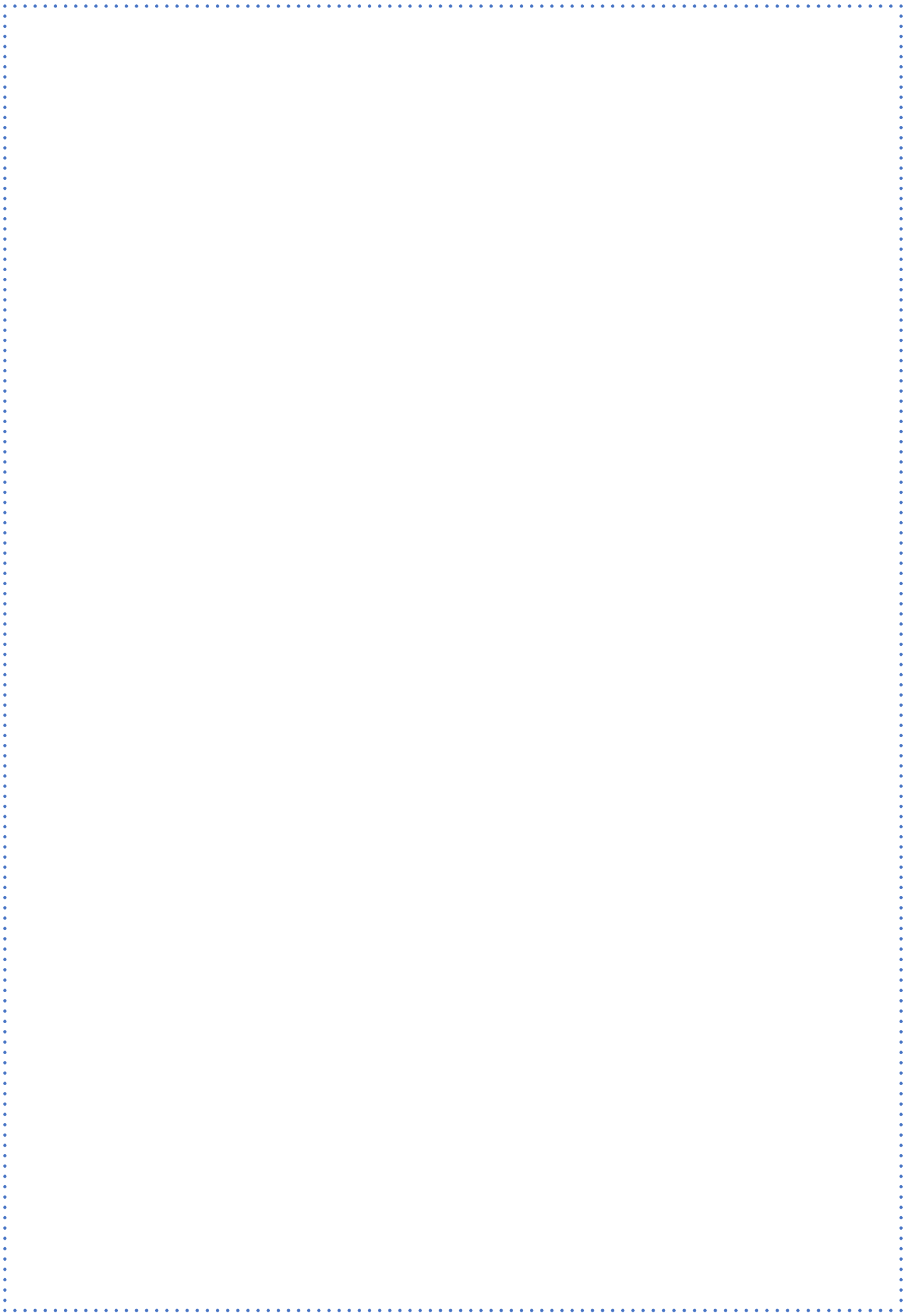
**M. EL MOSTAFA BELMEKKI**

**Professeur Investigation de malwares**

**Encadrement académique :**

**M. EL MOSTAFA BELMEKKI**

**Professeur Investigation de malwares**



Ministère de la Transition Numérique  
Et de l'Innovation Technologique

Ecole Supérieure Africaine des Technologies  
De l'Information et de la Communication



Année universitaire 2025-2026  
Institut National des Postes  
Et Communications



## **MÉMOIRE DE FIN DE CYCLE POUR L'OBTENTION DU DIPLÔME DE MASTER EN SÉCURITÉ INFORMATIQUE ET TECHNOLOGIES DU WEB (SITW)**

---

**SUJET : CONCEPTION ET MISE EN ŒUVRE D'UN FRAMEWORK  
INTELLIGENT DE DÉTECTION DES MALWARES ET ROOTKITS  
DANS LES ENVIRONNEMENTS CLOUD VIRTUALISÉS.**

---

Présenté par :

**Mlle Moayé Line Esther KOUASSI**

Période de stage :

**05 MARS 2026 – 10 JUILLET 2026**

**Maître de stage**

**M. EL MOSTAFA BELMEKKI**

**Professeur Investigation de malwares**

**Encadrement académique**

**M. EL MOSTAFA BELMEKKI**

**Professeur Investigation de malwares**

# DÉDICACE

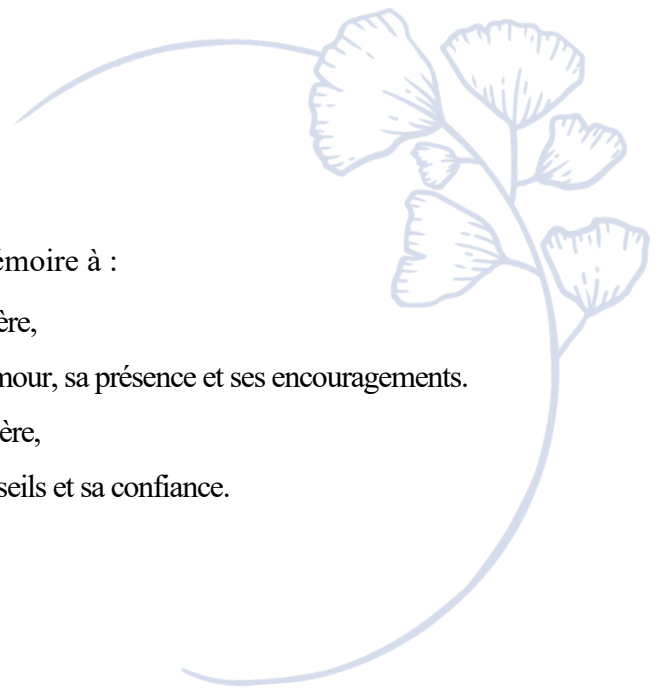
Je dédie ce mémoire à :

À ma mère,

Source de motivation, de courage, pour son amour, sa présence et ses encouragements.

À mon père,

Pour son soutien, ses conseils et sa confiance.



# REMERCIEMENTS

Je tiens à remercier chaleureusement toutes les personnes qui ont contribué à la réalisation de ce mémoire. Les conseils et encouragements reçus, tant sur le plan académique que professionnel, m'ont été d'une aide précieuse. Ces remerciements s'adressent notamment à :

- Aux directions de l'ESATIC et de l'INPT pour leur partenariat, grâce auquel j'ai bénéficié d'une année de mobilité internationale. Cette expérience a considérablement enrichi mon parcours académique et professionnel.
- Aux équipes pédagogiques de l'INPT pour la qualité de l'enseignement dispensé tout au long de ma formation.
- À Monsieur EL MOSTAFA BELMEKKI, encadrant académique et maître de stage à l'INPT, pour cette opportunité de stage, sa disponibilité et son accompagnement tout au long de cette période.
- À Monsieur HAFID BARKA et à Monsieur MOSTAFA BELLAFKIH d'avoir accepté de consacrer leur temps et leurs efforts à l'évaluation de ce travail en qualité de membres du jury.

Mes remerciements vont également à l'endroit de mes aînés issus des promotions précédentes, de ma famille et aussi de mes proches amis, en particulier à M. Hans Ariel DOH et Mlle Daniela KONAN, pour leur soutien moral et leur encouragement tout au long de cette aventure jalonnée de défis.

# RÉSUMÉ

Les mécanismes de dissimulation employés par les malwares et les rootkits modernes rendent leur identification particulièrement difficile lorsque l'intégrité du système ne peut plus être garantie. Afin de répondre à cette problématique, ce travail propose un framework de surveillance externe reposant sur l'introspection mémoire des machines virtuelles et l'apprentissage automatique non supervisé.

Le framework est déployé dans un environnement cloud virtualisé mono-nœud reposant sur la plateforme OpenStack, mise en œuvre à l'aide de DevStack. Les dumps mémoire sont acquis directement depuis l'hyperviseur, sans nécessiter l'installation d'un agent dans les machines virtuelles, puis analysés à l'aide de Volatility 3 afin d'en extraire des caractéristiques comportementales représentatives de l'état du système.

Ces caractéristiques extraites alimentent un détecteur basé sur One-Class Support Vector Machine, entraîné sans étiquetage sur un corpus majoritairement sain, dont il apprend le domaine de normalité. L'approche retenue combine ainsi les capacités d'observation offertes par l'introspection mémoire avec un modèle de détection d'anomalies capable d'identifier des comportements atypiques.

Les expérimentations montrent une détection fiable des systèmes compromis, un faible taux de faux positifs ainsi qu'une capacité à identifier des variantes de rootkits non observées durant l'entraînement. Elles mettent également en évidence le potentiel du framework pour détecter des menaces opérant aussi bien en espace noyau qu'en espace utilisateur, confirmant ainsi la pertinence des caractéristiques retenues et de l'approche proposée.

**Mots-clés :** détection de rootkits, OpenStack, DevStack, introspection mémoire, analyse forensique, apprentissage automatique non supervisé, détection d'anomalies, Volatility 3.

# ABSTRACT

The concealment mechanisms employed by modern malware and rootkits make their identification particularly difficult once the integrity of the system can no longer be guaranteed. To address this issue, this work proposes an external monitoring framework based on virtual machine memory introspection and unsupervised machine learning.

The framework is deployed in a single-node virtualized cloud environment based on the OpenStack platform, implemented using DevStack. Memory dumps are acquired directly from the hypervisor, without requiring the installation of an agent inside the virtual machines, and are then analyzed using Volatility 3 in order to extract behavioral features representative of the system state.

These extracted features feed a detector based on One-Class Support Vector Machine, trained without labels on a predominantly benign corpus, from which it learns the domain of normality. The chosen approach thus combines the observation capabilities offered by memory introspection with an anomaly detection model able to identify atypical behaviors.

The experiments show reliable detection of compromised systems, a low false positive rate, as well as an ability to identify rootkit variants not observed during training. They also highlight the potential of the framework to detect threats operating in kernel space as well as in user space, thereby confirming the relevance of the selected features and of the proposed approach.

**Keywords:** rootkit detection, OpenStack, DevStack, memory introspection, forensic analysis, unsupervised machine learning, anomaly detection, Volatility 3.

# TABLE DES FIGURES

|                                                                                          |    |
|------------------------------------------------------------------------------------------|----|
| Figure 1.1. Logo INPT .....                                                              | 3  |
| Figure 1.2. Organigramme interne INPT .....                                              | 4  |
| Figure 2.1. Schéma principe de virtualisation.....                                       | 8  |
| Figure 2.2. Catégories d'hyperviseurs .....                                              | 9  |
| Figure 2.3. Vecteurs d'attaques dans le cloud.....                                       | 11 |
| Figure 3.1. Hyperviseur Xen .....                                                        | 17 |
| Figure 3.2. Hyperviseur VMware ESXi.....                                                 | 18 |
| Figure 3.3. Hyperviseur KVM/QEMU .....                                                   | 18 |
| Figure 3.4. Logo OpenStack .....                                                         | 19 |
| Figure 3.5. Logo Volatility .....                                                        | 22 |
| Figure 3.6. Pipeline du module de détection.....                                         | 33 |
| Figure 3.7. Schéma représentatif du flux de données entre les modules du framework ..... | 34 |
| Figure 4.1. Vérification du support matériel de virtualisation .....                     | 36 |
| Figure 4.2. Environnement de virtualisation KVM/QEMU.....                                | 37 |
| Figure 4.3. Configuration de déploiement DevStack.....                                   | 37 |
| Figure 4.4. Dashboard OpenStack.....                                                     | 38 |
| Figure 4.5. Logo Python et Bash.....                                                     | 38 |
| Figure 4.6. VM Windows active sur OpenStack .....                                        | 39 |
| Figure 4.7. Présentation d'un cas de capture mémoire .....                               | 40 |
| Figure 4.8. Installation de Volatility .....                                             | 40 |
| Figure 4.9. Architecture du fonctionnement du framework de détection .....               | 42 |
| Figure 5.1. Présentation du pouvoir discriminant par caractéristique.....                | 49 |
| Figure 5.2. Résultats de l'expérience de sélection gloutonne.....                        | 52 |
| Figure 5.3. Résultats de métriques IF .....                                              | 53 |
| Figure 5.4. Résultats de métriques LOF .....                                             | 54 |
| Figure 5.5. Résultats de métriques OCSVM .....                                           | 55 |
| Figure 6.1. Résultats de détection état SAIN .....                                       | 59 |
| Figure 6.2. Activation du pilote ThrottleStop.sys .....                                  | 59 |
| Figure 6.3. Résultats de détection état INFECTED .....                                   | 60 |
| Figure 6.4. Illustration attaque DKOM .....                                              | 60 |
| Figure 6.5. Résultats détection DKOM .....                                               | 61 |



|                                                                                                               |      |
|---------------------------------------------------------------------------------------------------------------|------|
| Figure 6.6. Chargement rootkit userland.....                                                                  | 61   |
| Figure 6.7. Détection d'une menace userland en activité .....                                                 | 62   |
| Figure 6.8. Détails des artefacts userland détectés .....                                                     | 62   |
| Figure A.1. Diagramme de Gantt .....                                                                          | XVI  |
| Figure A.2. Scénario de malware en espace utilisateur par injection de la DLL<br>reflective_dll.x64.dll ..... | XVI  |
| Figure A.3. Pile d'expérimentations du détecteur.....                                                         | XVI  |
| Figure A.4. Résultats du test exact de Fisher sur la redondance du vecteur .....                              | XVII |
| Figure A.5. Résultats du test bootstrap sur l'écart de rappel .....                                           | XVII |

# LISTE DES TABLEAUX

|                                                                                           |    |
|-------------------------------------------------------------------------------------------|----|
| Tableau 3.1. Stratégies de capture mémoire .....                                          | 22 |
| Tableau 3.2. Plugins Volatility 3 utilisés pour l'extraction des caractéristiques .....   | 24 |
| Tableau 3.3. Indicateurs de dissimulation d'objets noyau .....                            | 25 |
| Tableau 3.4. Indicateurs de détournement et d'interception .....                          | 25 |
| Tableau 3.5. Indicateurs contextuels .....                                                | 26 |
| Tableau 3.6. Caractéristiques noyau retenues .....                                        | 27 |
| Tableau 3.7. Caractéristiques de détection userland.....                                  | 27 |
| Tableau 3.8. Tableau comparatif approche Machine Learning .....                           | 29 |
| Tableau 4.1. Caractéristiques de la machine hôte (nœud de supervision).....               | 35 |
| Tableau 5.1. Répartition du dataset .....                                                 | 45 |
| Tableau 5.2. Analyse des caractéristiques historiques.....                                | 46 |
| Tableau 5.3. Analyse des caractéristiques noyau .....                                     | 47 |
| Tableau 5.4. Analyse des caractéristiques de la couche espace utilisateur.....            | 48 |
| Tableau 5.5. Métriques d'évaluation .....                                                 | 51 |
| Tableau 5.6. Tableau récapitulatif de paramétrage IF .....                                | 52 |
| Tableau 5.7. Tableau récapitulatif de paramétrage LOF .....                               | 54 |
| Tableau 5.8. Tableau récapitulatif de paramétrage OCSVM.....                              | 55 |
| Tableau 5.9. Performances des trois modèles sur les trois jeux de caractéristiques.....   | 56 |
| Tableau 5.10. Paramètres de calibration du seuil de décision du modèle One-Class SVM..... | 57 |
| Tableau 6.1. Positionnement par rapport à l'état de l'art.....                            | 63 |

# LISTE DES ABRÉVIATIONS

|              |                                                              |
|--------------|--------------------------------------------------------------|
| <b>API</b>   | Application Programming Interface                            |
| <b>AUC</b>   | Area Under the Curve                                         |
| <b>CPU</b>   | Central Processing Unit                                      |
| <b>CSV</b>   | Comma-Separated Values                                       |
| <b>DKOM</b>  | Direct Kernel Object Manipulation                            |
| <b>DLL</b>   | Dynamic Link Library                                         |
| <b>FPR</b>   | False Positive Rate                                          |
| <b>IaaS</b>  | Infrastructure as a Service                                  |
| <b>IF</b>    | Isolation Forest                                             |
| <b>INPT</b>  | Institut National des Postes et Télécommunications           |
| <b>IRP</b>   | I/O Request Packet                                           |
| <b>KVM</b>   | Kernel-based Virtual Machine                                 |
| <b>LOF</b>   | Local Outlier Factor                                         |
| <b>LTSC</b>  | Long-Term Servicing Channel                                  |
| <b>MAD</b>   | Median Absolute Deviation                                    |
| <b>ML</b>    | Machine Learning                                             |
| <b>OCSVM</b> | One-Class Support Vector Machine                             |
| <b>OS</b>    | Operating System                                             |
| <b>PaaS</b>  | Platform as a Service                                        |
| <b>PDB</b>   | Program Database                                             |
| <b>QEMU</b>  | Quick Emulator                                               |
| <b>RAISS</b> | Réseaux, Architectures, Ingénierie des Services et Sécurités |
| <b>RAM</b>   | Random Access Memory (mémoire vive)                          |
| <b>ROC</b>   | Receiver Operating Characteristic                            |
| <b>SaaS</b>  | Software as a Service                                        |
| <b>SSDT</b>  | System Service Descriptor Table                              |

|             |                                                        |
|-------------|--------------------------------------------------------|
| <b>STRS</b> | Systèmes de Télécommunications, Réseaux et Services    |
| <b>SVM</b>  | Support Vector Machine (machine à vecteurs de support) |
| <b>TIC</b>  | Technologies de l'Information et de la Communication   |
| <b>TKRD</b> | Trusted Kernel Rootkit Detection                       |
| <b>TPR</b>  | True Positive Rate (taux de vrais positifs)            |
| <b>VM</b>   | Virtual Machine                                        |
| <b>VMI</b>  | Virtual Machine Introspection                          |
| <b>VMM</b>  | Virtual Machine Monitor                                |
| <b>VT-x</b> | Intel Virtualization Technology                        |

# TABLE DES MATIÈRES

|                                                                    |             |
|--------------------------------------------------------------------|-------------|
| <b>DÉDICACE.....</b>                                               | <b>I</b>    |
| <b>REMERCIEMENTS.....</b>                                          | <b>II</b>   |
| <b>RÉSUMÉ.....</b>                                                 | <b>III</b>  |
| <b>ABSTRACT .....</b>                                              | <b>IV</b>   |
| <b>TABLE DES FIGURES.....</b>                                      | <b>V</b>    |
| <b>LISTE DES TABLEAUX .....</b>                                    | <b>VII</b>  |
| <b>LISTE DES ABRÉVIATIONS .....</b>                                | <b>VIII</b> |
| <b>INTRODUCTION GÉNÉRALE .....</b>                                 | <b>1</b>    |
| <b>PREMIÈRE PARTIE :.....</b>                                      | <b>2</b>    |
| <b>CONTEXTE ET ÉTAT DE L'ART .....</b>                             | <b>2</b>    |
| <b>Chapitre 1 : Contexte général du projet .....</b>               | <b>3</b>    |
| <b>I. Présentation de la structure d'accueil .....</b>             | <b>3</b>    |
| 1. Historique et missions .....                                    | 3           |
| 2. Organisation interne et équipe de recherche .....               | 3           |
| <b>II. Présentation du projet .....</b>                            | <b>5</b>    |
| 1. Contexte du projet .....                                        | 5           |
| 2. Problématique.....                                              | 5           |
| 3. Objectifs du projet.....                                        | 6           |
| a. Objectif principal.....                                         | 6           |
| b. Objectifs spécifiques .....                                     | 6           |
| 4. Cahier des charges.....                                         | 6           |
| a. Exigences fonctionnelles.....                                   | 6           |
| b. Exigences non fonctionnelles.....                               | 7           |
| <b>Chapitre 2 : Fondements théoriques et travaux connexes.....</b> | <b>8</b>    |
| <b>I. Virtualisation et menaces associées .....</b>                | <b>8</b>    |
| 1. Concepts fondamentaux de la virtualisation .....                | 8           |
| a. Définition et principes de la virtualisation .....              | 8           |

|            |                                                                                                |           |
|------------|------------------------------------------------------------------------------------------------|-----------|
| b.         | Rôle de l'hyperviseur et mécanismes de virtualisation .....                                    | 9         |
| 2.         | Les environnements cloud virtualisés .....                                                     | 10        |
| a.         | Architecture des environnements cloud virtualisés.....                                         | 10        |
| b.         | Enjeux de sécurité dans les environnements cloud virtualisés.....                              | 10        |
| 3.         | Panorama des menaces.....                                                                      | 11        |
| a.         | Les malwares dans les environnements cloud.....                                                | 11        |
| b.         | Les rootkits : une menace avancée .....                                                        | 11        |
| c.         | Attaques spécifiques aux environnements virtualisés .....                                      | 12        |
| <b>II.</b> | <b>Analyse des approches existantes.....</b>                                                   | <b>12</b> |
| 1.         | Travaux existants sur la détection de malwares et rootkits en environnement<br>virtualisé..... | 12        |
| a.         | Détection basée sur les signatures et les mécanismes internes .....                            | 13        |
| b.         | Détection basée sur l'analyse mémoire .....                                                    | 13        |
| c.         | Détection basée sur l'introspection des machines virtuelles .....                              | 14        |
| d.         | Détection basée sur le Machine Learning .....                                                  | 14        |
| 2.         | Synthèse critique des approches étudiées.....                                                  | 15        |
|            | <b>DEUXIÈME PARTIE : .....</b>                                                                 | <b>16</b> |
|            | <b>CONCEPTION ET IMPLEMENTATION DU FRAMEWORK.....</b>                                          | <b>16</b> |
|            | <b>Chapitre 3 : Conception du framework .....</b>                                              | <b>17</b> |
| <b>I.</b>  | <b>Choix de l'environnement de virtualisation.....</b>                                         | <b>17</b> |
| 1.         | Choix de l'hyperviseur .....                                                                   | 17        |
| a.         | Xen .....                                                                                      | 17        |
| b.         | VMware ESXi .....                                                                              | 18        |
| c.         | KVM /QEMU.....                                                                                 | 18        |
| 2.         | Choix de la plateforme cloud .....                                                             | 19        |
| a.         | Présentation d'OpenStack .....                                                                 | 19        |
| b.         | Services OpenStack.....                                                                        | 20        |
| <b>II.</b> | <b>Architecture globale du framework .....</b>                                                 | <b>20</b> |
| 1.         | Définition d'un framework.....                                                                 | 20        |
| 2.         | Principe de conception du framework.....                                                       | 20        |
| a.         | Module d'introspection mémoire du framework .....                                              | 21        |
| b.         | Module d'analyse mémoire du framework .....                                                    | 22        |

|                                                                     |                                                               |           |
|---------------------------------------------------------------------|---------------------------------------------------------------|-----------|
| c.                                                                  | Module d'extraction de caractéristiques du framework.....     | 24        |
| d.                                                                  | Module de détection .....                                     | 28        |
| 3.                                                                  | Flux de données entre les modules.....                        | 34        |
| <b>Chapitre 4 : Implémentation des composants du framework.....</b> |                                                               | <b>35</b> |
| <b>I.</b>                                                           | <b>Environnement de développement et de déploiement .....</b> | <b>35</b> |
| 1.                                                                  | Environnement matériel .....                                  | 35        |
| 2.                                                                  | Environnement logiciel .....                                  | 35        |
| a.                                                                  | Mise en place de KVM/QEMU .....                               | 36        |
| b.                                                                  | Déploiement de DevStack/OpenStack .....                       | 37        |
| c.                                                                  | Outils et dépendances du framework .....                      | 38        |
| 3.                                                                  | Environnement d'expérimentation.....                          | 38        |
| <b>II.</b>                                                          | <b>Implémentation des modules du framework .....</b>          | <b>39</b> |
| 1.                                                                  | Module d'introspection mémoire.....                           | 39        |
| 2.                                                                  | Module d'analyse mémoire forensique.....                      | 40        |
| 3.                                                                  | Module d'extraction de caractéristiques .....                 | 41        |
| 4.                                                                  | Module de détection .....                                     | 41        |
| 5.                                                                  | Scheduler : surveillance continue.....                        | 41        |
| <b>TROISIÈME PARTIE : .....</b>                                     |                                                               | <b>43</b> |
| <b>ÉVALUATION EXPERIMENTALE ET RESULTATS .....</b>                  |                                                               | <b>43</b> |
| <b>Chapitre 5 : Dataset et évaluation des modèles .....</b>         |                                                               | <b>44</b> |
| <b>I.</b>                                                           | <b>Méthodologie expérimentale .....</b>                       | <b>44</b> |
| 1.                                                                  | Périmètre des menaces étudiées .....                          | 44        |
| 2.                                                                  | Construction du dataset CSV .....                             | 44        |
| <b>II.</b>                                                          | <b>Évaluation des caractéristiques extraites .....</b>        | <b>46</b> |
| 1.                                                                  | Analyse des premières caractéristiques.....                   | 46        |
| 2.                                                                  | Analyse des caractéristiques rootkit noyau .....              | 47        |
| 3.                                                                  | Analyse des caractéristiques userland .....                   | 48        |
| 4.                                                                  | Analyse globale du jeu de données .....                       | 49        |
| <b>III.</b>                                                         | <b>Évaluation des modèles de détection .....</b>              | <b>50</b> |

|                                                                 |            |
|-----------------------------------------------------------------|------------|
| 1. Isolation Forest .....                                       | 52         |
| a. Paramétrage et entraînement .....                            | 52         |
| b. Résultats obtenus .....                                      | 53         |
| 2. Local Outlier Factor .....                                   | 53         |
| a. Paramétrage et entraînement .....                            | 53         |
| b. Résultats obtenus et analyse .....                           | 54         |
| 3. One-Class Support Vector Machine .....                       | 55         |
| a. Paramétrage et entraînement .....                            | 55         |
| b. Résultats obtenus .....                                      | 55         |
| 4. Analyse comparative et composition finale du détecteur ..... | 56         |
| <b>Chapitre 6 : Résultats et analyse .....</b>                  | <b>58</b>  |
| <b>I. Expérimentation du détecteur final .....</b>              | <b>58</b>  |
| 1. Détection de rootkits noyau .....                            | 58         |
| a. Détection de la présence d'un rootkit chargé .....           | 58         |
| b. Détection comportementale .....                              | 60         |
| 2. Détection de menaces en espace utilisateur .....             | 61         |
| <b>II. Analyse comparative des résultats .....</b>              | <b>63</b>  |
| 1. Comparaison avec l'approche TKRD .....                       | 63         |
| 2. Positionnement du framework .....                            | 64         |
| <b>III. Discussion critique .....</b>                           | <b>64</b>  |
| 1. Points forts du framework .....                              | 64         |
| 2. Positionnement scientifique du travail .....                 | 65         |
| 3. Limites de l'approche .....                                  | 66         |
| 2. Perspectives .....                                           | 67         |
| <b>CONCLUSION GÉNÉRALE .....</b>                                | <b>69</b>  |
| <b>RÉFÉRENCES .....</b>                                         | <b>XIV</b> |
| <b>ANNEXE .....</b>                                             | <b>XVI</b> |



# INTRODUCTION GÉNÉRALE

Les malwares constituent aujourd'hui l'une des principales menaces pesant sur les systèmes informatiques. Parmi eux, les rootkits occupent une place particulière en raison de leur capacité à dissimuler leur présence et leurs activités au sein d'un système compromis. En s'intégrant profondément au système d'exploitation, ils peuvent contourner les mécanismes de surveillance interne, masquer certains artefacts de leur activité et compromettre la fiabilité des informations collectées localement.

Cette capacité de furtivité rend leur détection particulièrement complexe, notamment dans les environnements virtualisés et cloud où les machines virtuelles hébergent souvent des services et des données critiques.

Face à ces limitations, de nouvelles approches de détection ont émergé, parmi lesquelles l'introspection mémoire des machines virtuelles. En contournant le système potentiellement compromis, elle offre une meilleure fiabilité pour l'analyse forensique et la détection d'activités malveillantes. Parallèlement, les techniques d'apprentissage automatique offrent la possibilité d'identifier automatiquement des comportements anormaux et de détecter des menaces inédites ou évolutives.

C'est dans ce cadre que s'inscrit le présent travail, intitulé « Conception et mise en œuvre d'un framework intelligent de détection des malwares et rootkits dans les environnements cloud virtualisés ». La problématique centrale abordée est la suivante : **Comment exploiter efficacement l'introspection mémoire combinée au Machine Learning pour détecter les compromissions des machines virtuelles ?**

Pour y répondre, une solution basée sur l'utilisation conjointe de l'outil d'analyse mémoire et de modèles de Machine Learning entraînés sur des caractéristiques extraites dynamiquement est proposée. L'objectif est de concevoir un système capable de détecter de manière fiable des compromissions associées à des malwares et rootkits, y compris certaines variantes non observées durant l'entraînement.

Ce mémoire s'articule en six chapitres. Le premier présente le contexte du projet, tandis que le deuxième est consacré aux fondements de la virtualisation, de l'analyse mémoire et à l'état de l'art. Les chapitres suivants décrivent la conception du framework, son implémentation, puis la validation expérimentale des choix de conception et enfin les résultats obtenus et leur analyse.

# PREMIÈRE PARTIE :

## CONTEXTE ET ÉTAT DE L'ART

Cette partie présente le cadre général du projet, son contexte, ses objectifs et son cahier des charges. Elle introduit également les fondements de la virtualisation, les menaces dans les environnements cloud et l'état de l'art des approches de détection.

# Chapitre 1 : Contexte général du projet

## Introduction

Ce chapitre présente le cadre général du projet : la structure d'accueil, le contexte, les objectifs et le cahier des charges qui ont orienté l'ensemble du travail.

## I. Présentation de la structure d'accueil

### 1. Historique et missions

L'Institut National des Postes et Télécommunications (INPT) est un établissement public d'enseignement supérieur et de recherche relevant du secteur des technologies de l'information et de la communication au Maroc. Créé en **1961** sous la tutelle du **ministère chargé des télécommunications**, il avait initialement pour mission de former les cadres destinés au secteur des postes et télécommunications. Au fil de son évolution, l'institut a progressivement élargi ses domaines de compétence pour devenir une référence nationale dans la formation d'ingénieurs, la recherche appliquée et l'innovation dans les domaines des télécommunications, des systèmes d'information et des technologies numériques.



Figure 1.1. Logo INPT

Dans le cadre de ses activités, l'institut poursuit plusieurs objectifs fondamentaux :

- Assurer une formation scientifique et technique de haut niveau ;
- Développer la recherche appliquée dans les domaines des TIC ;
- Favoriser l'innovation et le transfert technologique vers le secteur socio-économique ;
- Accompagner la transformation numérique des organisations publiques et privées.

### 2. Organisation interne et équipe de recherche

L'organisation de l'INPT repose sur une structure académique et scientifique articulée autour de départements d'enseignement et de laboratoires de recherche.

Les activités de recherche sont principalement regroupées au sein de laboratoires spécialisés, dont le laboratoire **STRS** (Systèmes de Télécommunications, Réseaux et Services), qui constitue l'un des principaux pôles de recherche de l'institut. Ce laboratoire regroupe plusieurs équipes travaillant sur des thématiques complémentaires telles que :

- Les réseaux et systèmes de communication ;
- Les systèmes distribués et le cloud computing ;
- L'intelligence artificielle et l'analyse de données ;
- La cybersécurité et la protection des systèmes informatiques.

Ces équipes collaborent afin de répondre aux problématiques scientifiques et industrielles liées aux infrastructures numériques modernes, notamment la sécurité des environnements virtualisés et cloud.

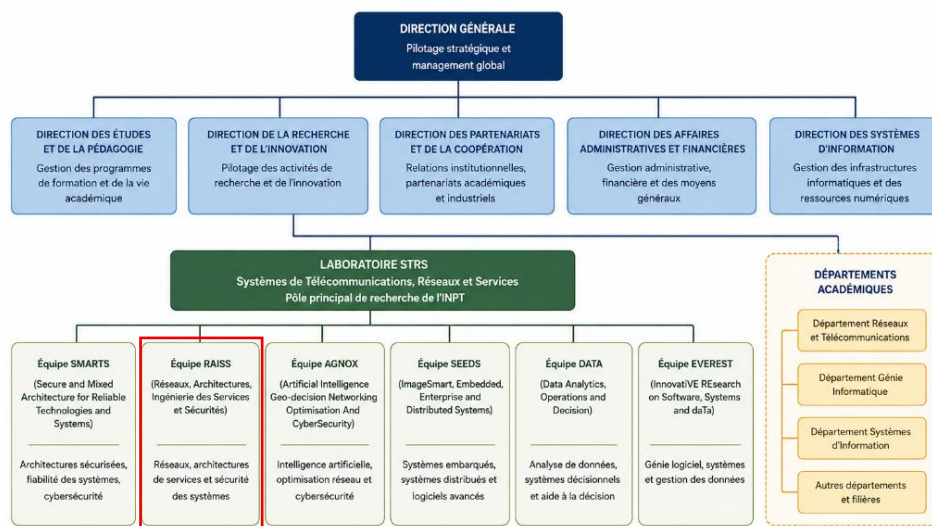


Figure 1.2. Organigramme interne INPT

Dans le cadre de ce projet, les travaux s'inscrivent principalement dans les axes de recherche de l'équipe **RAISS** (Réseaux, Architectures, Ingénierie des Services et Sécurités) du laboratoire STRS (voir Figure 1.2). Ils portent sur la cybersécurité des environnements virtualisés, en particulier sur les problématiques de cloud computing, de virtualisation et de sécurité des systèmes. Notre travail s'inscrit dans ces thématiques en s'intéressant à la détection de menaces avancées, notamment les malwares et les rootkits, au moyen de l'introspection mémoire et de techniques d'apprentissage automatique.

La conduite de ce travail s'est étalée sur toute la durée du stage, selon une planification détaillée en annexe (voir Annexe, Figure A.1)

## II. Présentation du projet

### 1. Contexte du projet

Dans le cadre des activités de l'équipe de recherche **RAISS**, la détection des menaces dans les environnements cloud et virtualisés constitue un enjeu majeur. Ces environnements, bien que largement adoptés, sont exposés à des attaques avancées telles que les malwares et les rootkits, souvent difficiles à détecter par les approches classiques.

Face à ces limites, l'introspection mémoire apparaît comme une alternative pertinente, permettant d'analyser une machine virtuelle depuis une couche externe de confiance.

Notre projet s'inscrit dans cette démarche, en proposant la conception d'un framework intelligent de détection adapté aux environnements cloud virtualisés.

### 2. Problématique

Les rootkits représentent une menace particulièrement critique dans les environnements cloud virtualisés : leur capacité à s'intégrer profondément au système compromis leur permet de dissimuler leur présence et de falsifier les informations fournies aux outils de surveillance internes.

Les approches de détection traditionnelles, fondées sur des signatures ou des mécanismes internes au système observé, se révèlent donc structurellement limitées face à ce type de menace, puisqu'elles dépendent précisément des couches que l'attaquant cherche à compromettre. L'introspection mémoire des machines virtuelles constitue une réponse à cette limitation, en permettant d'observer l'état d'une VM depuis une couche externe de confiance, indépendante du système potentiellement compromis. Toutefois, cette approche seule ne suffit pas : elle produit un volume important de données brutes qu'il est nécessaire d'interpréter, et ne permet pas, à elle seule, de distinguer automatiquement un comportement légitime d'un comportement malveillant, en particulier face à des menaces inédites ou évolutives.

Ces constats soulèvent la problématique centrale de ce travail : **comment exploiter efficacement l'introspection mémoire combinée au Machine Learning pour détecter, de manière fiable et non intrusive, les compromissions des machines virtuelles dans un**

**environnement cloud virtualisé ?** Cette problématique se décline en trois questions de recherche :

- Comment observer une machine virtuelle sans dépendre de son système interne, potentiellement compromis ?
- Quelles caractéristiques extraites de la mémoire permettent d'identifier un comportement malveillant, y compris pour des menaces non répertoriées ?
- Comment concevoir un mécanisme de détection par apprentissage automatique capable de généraliser à des menaces non vues lors de l'entraînement, tout en limitant les faux positifs ?

### 3. Objectifs du projet

#### a. Objectif principal

L'objectif principal de ce projet est de concevoir et de mettre en œuvre un framework de détection des malwares et des rootkits dans un environnement cloud virtualisé, en s'appuyant sur l'introspection mémoire et des techniques d'apprentissage automatique ou Machine Learning en anglais, afin d'identifier des comportements et/ou présences suspects de manière fiable et non intrusive.

#### b. Objectifs spécifiques

Pour atteindre cet objectif, il est prévu de mettre en place un environnement cloud virtualisé permettant le déploiement de machines virtuelles cibles ainsi que l'implémentation d'un mécanisme d'introspection mémoire destiné à collecter les données nécessaires à l'analyse. À partir de ces informations, un module de détection fondé sur des techniques de Machine Learning sera développé afin d'identifier les compromissions mémoire, puis son efficacité sera évaluée au travers de différents scénarios d'attaque représentatifs.

### 4. Cahier des charges

Les exigences fonctionnelles et non fonctionnelles du framework sont formalisées dans le cahier des charges, qui sert de référence pour orienter sa conception et son développement.

#### a. Exigences fonctionnelles

Le système doit permettre le déploiement et la gestion de machines virtuelles au sein d'un environnement cloud virtualisé, ainsi que la réalisation de captures mémoire directement depuis l'hyperviseur. Les données collectées doivent ensuite être exploitées afin d'extraire et

d'analyser des informations pertinentes pour la détection de comportements ou de présences malveillantes. Le système doit également permettre la mise en œuvre de scénarios d'attaque destinés à évaluer son efficacité, tout en fournissant les résultats de l'analyse sous forme d'alertes et de journaux d'événements facilitant leur exploitation par l'utilisateur.

b. Exigences non fonctionnelles

Au-delà des fonctionnalités attendues, le système doit également satisfaire plusieurs exigences non fonctionnelles. Il doit notamment fonctionner de manière non intrusive, sans nécessiter l'installation d'un agent au sein des machines virtuelles analysées, tout en restant compatible avec un environnement disposant de ressources limitées. Son architecture doit être modulaire afin de faciliter son évolution et l'intégration de nouveaux mécanismes de détection. Enfin, le système doit maintenir des performances compatibles avec l'analyse mémoire tout en garantissant une présentation claire et facilement interprétable des résultats.

Conclusion

Ce chapitre a permis de définir le périmètre du projet et d'établir les exigences auxquelles le système doit répondre. Ces éléments constituent le point de départ de l'étude approfondie menée dans le chapitre suivant.

# Chapitre 2 : Fondements théoriques et travaux connexes

## Introduction

Ce chapitre pose les bases théoriques du projet. Il présente les concepts fondamentaux de la virtualisation, les menaces associées aux environnements cloud, puis dresse un état de l'art des approches de détection existantes afin de positionner et justifier la solution proposée.

## I. Virtualisation et menaces associées

### 1. Concepts fondamentaux de la virtualisation

#### a. Définition et principes de la virtualisation

La **virtualisation** est une technique qui consiste à abstraire les ressources matérielles d'un système informatique afin de permettre l'exécution de plusieurs environnements indépendants sur une même machine physique [1]. Concrètement, elle permet de créer plusieurs machines virtuelles ou Virtual Machines (VM) en anglais, chacune disposant de son propre système d'exploitation et fonctionnant comme une machine autonome, alors qu'elles partagent en réalité les mêmes ressources matérielles. Cette abstraction repose sur l'idée de dissocier la couche matérielle de la couche logicielle. Ainsi, les applications et les systèmes d'exploitation ne sont plus directement dépendants du matériel physique, ce qui facilite leur déploiement, leur migration et leur gestion.

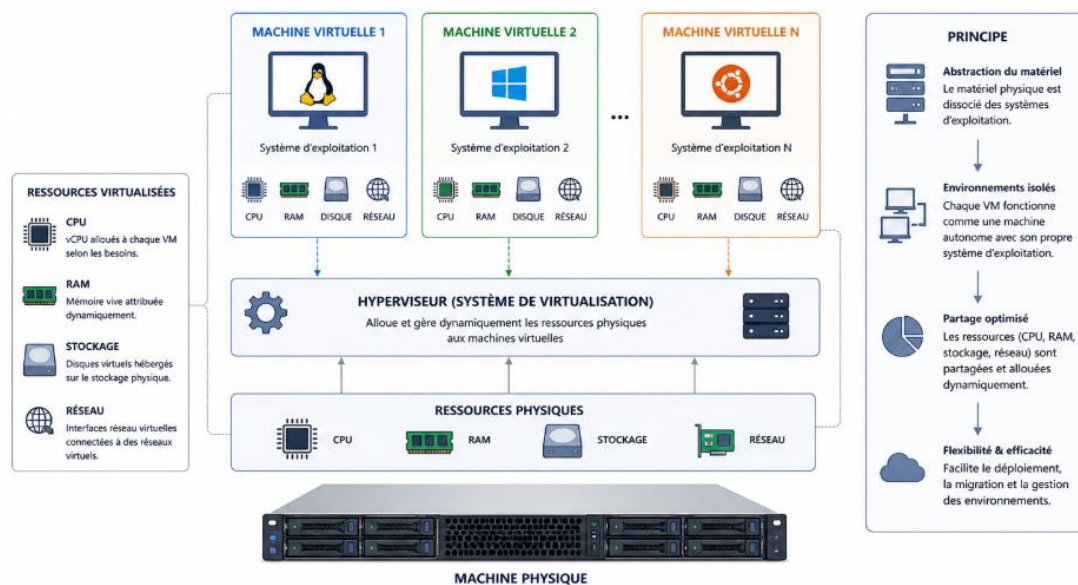


Figure 2.1. Schéma principe de virtualisation



b. Rôle de l'hyperviseur et mécanismes de virtualisation

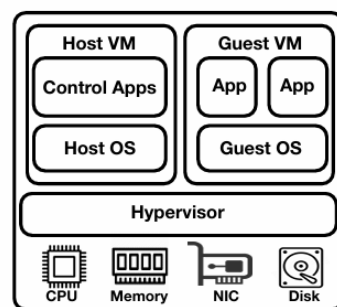
Le fonctionnement de la virtualisation repose sur un composant central appelé hyperviseur, également connu sous le nom de Virtual Machine Monitor (VMM) en anglais. Ce logiciel agit comme une couche intermédiaire entre le matériel physique et les machines virtuelles [2]. Il introduit ainsi une couche de virtualisation qui permet à plusieurs systèmes d'exploitation de coexister sur une même infrastructure tout en maintenant l'illusion d'un accès exclusif aux ressources.

L'hyperviseur assure plusieurs fonctions essentielles :

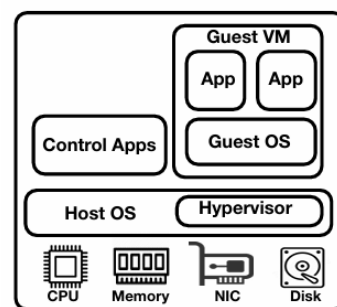
- **Gestion des ressources** : il répartit les ressources matérielles entre les différentes machines virtuelles en fonction de leurs besoins ;
- **Isolation** : il garantit que chaque machine virtuelle fonctionne dans un environnement isolé, empêchant les interférences ou les accès non autorisés entre elles ;
- **Contrôle du cycle de vie** : il permet de créer, démarrer, suspendre, migrer ou supprimer des machines virtuelles ;
- **Surveillance et sécurité** : il joue un rôle important dans la détection d'anomalies et la gestion des accès.

Par ailleurs, les hyperviseurs sont classés en deux catégories :

- **Hyperviseur de type 1 (bare-metal)** : exécuté directement sur le matériel, il offre de meilleures performances et un niveau de sécurité plus élevé.
- **Hyperviseur de type 2 (hosted)** : exécuté au-dessus d'un système d'exploitation hôte, il est plus simple à mettre en œuvre mais moins performant.



(a) Type-1 bare-metal virtualization platform



(b) Type-2 hosted virtualization platform

Figure 2.2. Catégories d'hyperviseurs

## 2. Les environnements cloud virtualisés

### a. Architecture des environnements cloud virtualisés

Les environnements cloud virtualisés reposent sur une architecture en couches permettant d'abstraire et de gérer efficacement les ressources informatiques [3] . Cette architecture comprend généralement plusieurs niveaux :

- **La couche matérielle** : constituée des serveurs physiques, du stockage et des équipements réseau hébergés dans des centres de données ;
- **La couche de virtualisation** : assurée par l'hyperviseur, qui permet de créer et de gérer les machines virtuelles ;
- **La couche des machines virtuelles** : où s'exécutent les systèmes d'exploitation invités et les applications ;
- **La couche des services cloud** : qui fournit des services aux utilisateurs (IaaS, PaaS, SaaS).

Dans ce type d'environnement, plusieurs machines virtuelles appartenant à différents utilisateurs peuvent coexister sur un même serveur physique. Cette organisation permet d'optimiser l'utilisation des ressources tout en offrant une grande flexibilité dans le déploiement des services.

### b. Enjeux de sécurité dans les environnements cloud virtualisés

Les environnements cloud virtualisés présentent des défis spécifiques en matière de sécurité, liés à leur architecture distribuée et à la mutualisation des ressources [4] . Parmi les principaux enjeux, on peut citer :

- **L'isolation entre machines virtuelles** : une faille dans l'hyperviseur ou dans les mécanismes d'isolation peut permettre à un attaquant d'accéder à d'autres machines virtuelles ;
- **La surface d'attaque étendue** : la présence de multiples couches (réseau, hyperviseur, VM) augmente les points d'entrée potentiels pour les attaquants ;
- **La gestion des accès** : les environnements cloud nécessitent des mécanismes d'authentification et de contrôle d'accès robustes ;

- **La protection des données** : les données stockées et traitées dans le cloud doivent être protégées contre les accès non autorisés et les fuites d'information.

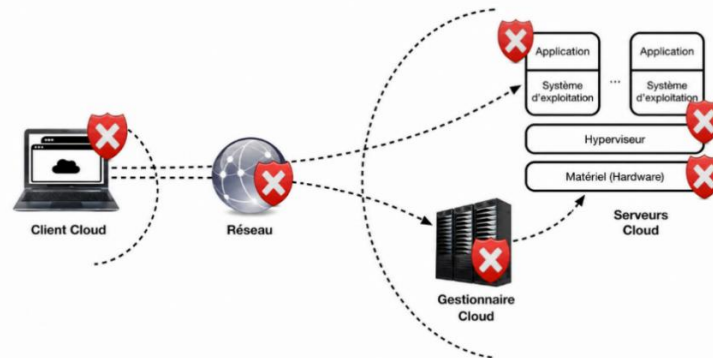


Figure 2.3. Vecteurs d'attaques dans le cloud

### 3. Panorama des menaces

Les environnements cloud virtualisés, en raison de leur complexité et de leur nature distribuée, sont exposés à une large variété de menaces [5]. La mutualisation des ressources, la coexistence de multiples machines virtuelles et la dépendance à des couches logicielles intermédiaires augmentent la surface d'attaque et introduisent de nouvelles vulnérabilités.

#### a. Les malwares dans les environnements cloud

Les malwares désignent l'ensemble des logiciels malveillants conçus pour infiltrer, perturber ou exploiter un système informatique. Dans les environnements cloud, ils peuvent être introduits à travers différentes sources, telles que des applications compromises, des fichiers infectés ou l'exploitation de vulnérabilités [6].

On distingue plusieurs catégories de malwares, notamment les virus, les vers (worms), les chevaux de Troie (Trojans), les spywares, les stealers ou encore les ransomwares. Toutefois, tous ne présentent pas nécessairement des artefacts mémoire exploitables pour une détection fondée sur l'introspection mémoire. Les approches de détection basées sur l'analyse mémoire sont particulièrement adaptées aux menaces qui modifient l'état du système, exécutent du code en mémoire ou interagissent avec des structures du système d'exploitation.

#### b. Les rootkits : une menace avancée

Parmi les différentes formes de malwares, les rootkits représentent une menace particulièrement critique [7]. Leur objectif principal est de maintenir un accès persistant au système tout en dissimulant leur présence.

Les rootkits peuvent opérer à différents niveaux :

- **Rootkits en espace utilisateur (userland)** : ils ciblent les applications et les processus en modifiant leur comportement. Ils s'exécutent en **mode utilisateur (Ring 3)**, c'est-à-dire au niveau d'exécution réservé aux applications, sans accès direct aux ressources critiques du noyau et utilisent généralement des techniques d'injection ou d'interception d'API afin de masquer certaines activités ou ressources du système ;
- **Rootkits en espace noyau (kernel-land)** : ils s'exécutent en mode **noyau (Ring 0)**, c'est-à-dire au niveau de privilège le plus élevé du processeur. Ils disposent ainsi d'un accès direct aux ressources critiques du système et peuvent interagir avec les structures internes afin de modifier ou de dissimuler leur fonctionnement.

c. Attaques spécifiques aux environnements virtualisés

En plus des menaces classiques, les environnements cloud virtualisés sont exposés à des attaques liées à leur architecture:

- **Attaques inter-VM** : un attaquant peut tenter d'exploiter une faille d'isolation pour accéder à d'autres machines virtuelles partageant le même hôte physique ;
- **Attaques sur l'hyperviseur** : la compromission de l'hyperviseur peut conférer un contrôle total sur l'ensemble des machines virtuelles hébergées ;
- **Attaques par canaux auxiliaires** (side-channel attacks) : elles exploitent des informations indirectes telles que le temps d'exécution ou la consommation de cache CPU pour extraire des données sensibles d'une VM voisine ;
- **Fuites de données** : dues à une mauvaise configuration ou à des failles dans la gestion des ressources partagées.

Ces attaques tirent directement parti des caractéristiques de la virtualisation, ce qui les rend particulièrement difficiles à détecter avec des approches classiques internes au système surveillé.

## II. Analyse des approches existantes

### 1. Travaux existants sur la détection de malwares et rootkits en environnement virtualisé

La détection des malwares et des rootkits constitue depuis plusieurs années un domaine majeur de recherche en cybersécurité.

L'évolution constante des techniques de furtivité utilisées par les attaquants a progressivement révélé les limites des approches traditionnelles de protection, notamment dans les environnements cloud et virtualisés [8] . [8] [8] Afin de répondre à ces nouveaux défis, plusieurs axes de recherche ont émergé, reposant notamment sur l'analyse mémoire, l'introspection des machines virtuelles et les techniques d'apprentissage automatique.

a. Détection basée sur les signatures et les mécanismes internes

Les premières solutions de détection reposent principalement sur des bases de signatures, des listes de réputation ou des mécanismes internes au système d'exploitation. Les antivirus traditionnels comparent les fichiers, processus ou activités observés à des modèles préalablement connus afin d'identifier la présence d'une menace. Cette approche présente l'avantage d'offrir une détection rapide et généralement précise pour les menaces déjà référencées. Toutefois, son efficacité dépend fortement de la mise à jour régulière des bases de signatures. Les variantes inconnues, polymorphes ou spécifiquement développées pour contourner les solutions de sécurité peuvent ainsi échapper à la détection. Cette limitation devient particulièrement critique dans le cas des rootkits. En opérant au sein même du système compromis, ces derniers peuvent modifier les mécanismes utilisés par les outils de sécurité pour observer le système. Les informations fournies aux antivirus ou aux outils d'administration peuvent alors être falsifiées, réduisant considérablement la fiabilité des analyses réalisées depuis l'intérieur de la machine surveillée. Ces limites ont conduit la communauté scientifique à explorer des approches capables d'observer le système depuis une couche extérieure de confiance.

b. Détection basée sur l'analyse mémoire

L'analyse mémoire constitue aujourd'hui l'une des approches les plus utilisées pour l'identification des menaces avancées. Contrairement aux analyses basées uniquement sur le système de fichiers, elle permet d'accéder directement à l'état réel du système au moment de la capture. La mémoire vive contient en effet de nombreuses informations telles que les processus actifs, les modules chargés, les connexions réseaux, les objets noyau ou encore les structures internes du système d'exploitation. Ces informations constituent des sources précieuses pour l'identification des comportements malveillants [9] . Toutefois, cette approche à elle seule présente certaines limites. La quantité d'informations extraite est souvent très importante et nécessite une expertise approfondie pour être interprétée correctement. De plus, la majorité des

outils d'analyse mémoire produisent essentiellement des informations brutes, laissant à l'analyste la responsabilité d'identifier manuellement les comportements anormaux.

Ces observations ont conduit à l'intégration progressive de mécanismes d'automatisation capables d'exploiter ces données à grande échelle.

c. Détection basée sur l'introspection des machines virtuelles

L'intérêt principal de l'introspection mémoire des machines virtuelles (Virtual Machine Introspection, VMI) réside dans son caractère out-of-the-box. L'observateur étant situé à l'extérieur de la machine virtuelle, un malware ou un rootkit ne peut généralement pas compromettre directement le mécanisme de surveillance. Les travaux de Zhang et Lee, à travers l'architecture CloudMonAtt [10], illustrent cet avantage en proposant un mécanisme d'attestation fondé sur des propriétés de sécurité vérifiées à partir des informations collectées depuis l'hyperviseur. Cette approche permet de surveiller l'intégrité des machines virtuelles sans dépendre des mécanismes internes potentiellement compromis. Toutefois, la VMI reste confrontée au problème du semantic gap, les données mémoire brutes devant être reconstruites afin de retrouver les structures logiques du système d'exploitation avant toute analyse.

d. Détection basée sur le Machine Learning

L'intégration du Machine Learning dans la détection de malwares/rootkits a permis de dépasser les limites des approches basées sur des signatures. En analysant des caractéristiques extraites du système (mémoire, appels système, activités réseau), les modèles d'apprentissage automatique peuvent identifier des comportements anormaux, y compris ceux associés à des menaces inconnues. Les travaux de Wang et al. sur la méthode **TKRD** (Trusted Kernel Rootkit Detection) [11] ont montré que la combinaison de l'analyse mémoire et de classifieurs d'apprentissage supervisé permet d'atteindre des performances élevées sur la détection de rootkits noyau dans des VMs de cloud privé. Les auteurs extraient des caractéristiques à partir de dumps mémoire puis comparent plusieurs algorithmes (arbres de décision, classifieurs à base de règles, classifieurs bayésiens, SVM, Random Forest) ; le Random Forest obtient les meilleurs résultats avec une accuracy de 0,986 et une AUC de 0,998 sur leur jeu de données expérimental, y compris sur des rootkits non vus à l'entraînement. Ces résultats, obtenus sur un environnement Windows 7 x86 au sein d'un cloud privé homogène, confirment l'intérêt de l'analyse mémoire et du Machine Learning.

## 2. Synthèse critique des approches étudiées

L'analyse de la littérature met en évidence plusieurs constats importants. D'une part, les approches basées sur les signatures demeurent efficaces contre les menaces connues, mais montrent rapidement leurs limites face aux variantes émergentes et aux techniques modernes de furtivité. D'autre part, l'analyse mémoire fournit une source d'information particulièrement riche pour la détection des rootkits, à condition de disposer de mécanismes capables d'interpréter automatiquement les données extraites. La Virtual Machine Introspection (VMI) quant à elle permet d'observer le système de manière fiable depuis l'extérieur de la machine virtuelle, en complément avec le Machine Learning qui automatise l'analyse des données mémoire afin d'identifier des comportements suspects ne correspondant pas nécessairement à des signatures connues. Toutefois, plusieurs limites subsistent. L'apprentissage supervisé nécessite la constitution de jeux de données étiquetés comportant à la fois des observations saines et un ensemble représentatif de scénarios d'attaque, souvent difficiles à collecter face à des menaces en constante évolution. Par ailleurs, l'interprétabilité des décisions produites reste limitée pour plusieurs modèles, et l'acquisition mémoire, l'analyse forensique et la détection automatique demeurent fréquemment traitées comme des étapes disjointes dans les travaux existants.

Notre contribution s'inscrit dans cette continuité en proposant un framework unifié intégrant l'acquisition des dumps mémoire, l'analyse forensique et la détection automatique par apprentissage automatique. Le framework a également été conçu pour être évalué sur un environnement récent, au moyen de scénarios mettant en œuvre des menaces contemporaines, afin d'améliorer la représentativité des expérimentations.

### Conclusion

Ce chapitre a mis en évidence les limites des approches classiques face aux menaces avancées et confirmé la pertinence de combiner introspection mémoire et apprentissage automatique. Ces enseignements guideront directement la conception du framework présentée dans la partie suivante.

## DEUXIÈME PARTIE : CONCEPTION ET IMPLÉMENTATION DU FRAMEWORK

Cette deuxième partie est consacrée à la conception du framework proposé et à sa mise en œuvre technique. Elle présente l'architecture globale du système, les choix technologiques retenus, ainsi que l'implémentation des différents modules qui composent le framework.



## Chapitre 3 : Conception du framework

### Introduction

Ce chapitre présente le principe de conception du framework proposé. Il expose les choix technologiques retenus et décrit l'architecture du système, de l'acquisition des données mémoire à leur exploitation par des modèles d'apprentissage automatique.

### I. Choix de l'environnement de virtualisation

#### 1. Choix de l'hyperviseur

Le choix de l'hyperviseur est une décision structurante pour l'ensemble du framework, car il conditionne directement la faisabilité des mécanismes d'introspection mémoire. Dans ce projet, l'étude s'est limitée aux hyperviseurs de type 1 (bare-metal), particulièrement adaptés aux environnements cloud et aux approches de surveillance externe. Trois solutions ont ainsi été étudiées et comparées :

##### a. Xen

L'hyperviseur **Xen** [12] est largement utilisé dans les environnements virtualisés et les travaux de recherche sur la VMI. Grâce à son architecture conçue pour la virtualisation, il facilite l'observation des machines virtuelles depuis une couche externe de confiance.

Cette caractéristique explique son utilisation dans plusieurs travaux de référence dédiés à la détection de menaces dans les environnements virtualisés tels que CloudMonAtt [10] . Toutefois, sa mise en œuvre et son administration sont généralement plus complexes que celles d'autres et son déploiement nécessite généralement davantage de ressources matérielles que certaines solutions de virtualisation plus légères.



Figure 3.1. Hyperviseur Xen

### b. VMware ESXi

De son côté, **VMware ESXi (Elastic Sky X Integrated)** [13] est un hyperviseur généralement utilisé dans l'industrie. Il se distingue par ses performances élevées, sa stabilité et ses fonctionnalités avancées de gestion des infrastructures virtualisées. Toutefois, en tant que solution propriétaire, il limite l'accès aux couches basses du système, notamment aux structures mémoire internes des machines virtuelles. Cette restriction constitue un obstacle majeur pour les approches basées sur l'introspection, qui nécessitent un accès fin aux ressources.



Figure 3.2. Hyperviseur VMware ESXi

### c. KVM /QEMU

**KVM (Kernel-based Virtual Machine)** [14] est un hyperviseur intégré au noyau Linux, introduit à partir de la version 2.6.20. Grâce à son caractère open source, sa flexibilité et son intégration avec l'écosystème Linux, il est fortement utilisé dans les infrastructures cloud modernes. Son association avec **QEMU (Quick EMUlator)**, permet de gérer efficacement les machines virtuelles tout en restant compatible avec les mécanismes d'introspection mémoire exploités dans ce projet.



Figure 3.3. Hyperviseur KVM/QEMU

Bien que Xen demeure une référence dans les architectures VMI avancées, l'association **KVM/QEMU** apparaît comme la solution la mieux adaptée aux contraintes du projet. Elle offre un compromis pertinent entre accessibilité, capacités d'introspection et compatibilité avec l'écosystème open source, sans nécessiter la complexité de mise en œuvre de Xen ni les restrictions des solutions propriétaires de VMware ESXi. La capture des dumps mémoire peut y être réalisée de manière non intrusive via les interfaces standards de QEMU, ce qui satisfait directement l'exigence du cahier des charges.

## 2. Choix de la plateforme cloud

La mise en œuvre du framework s'inscrit dans un contexte d'infrastructures virtualisées impliquant la gestion des machines virtuelles, du réseau et du stockage. Plusieurs plateformes cloud, telles qu'OpenStack, Apache CloudStack ou OpenNebula, permettent aujourd'hui de déployer et d'administrer de tels environnements. Cependant, parmi les différentes solutions envisageables, OpenStack a été retenu pour ce projet en raison de sa large adoption et de son adéquation avec la mise en place d'un environnement expérimental représentatif des infrastructures cloud modernes.

### a. Présentation d'OpenStack

**OpenStack** [15] est une plateforme open source de type Infrastructure as a Service (IaaS) utilisée pour le déploiement et l'administration d'environnements cloud publics, privés ou hybrides. Elle agit comme un système d'exploitation du cloud en assurant la gestion centralisée des ressources d'infrastructure telles que le calcul, le réseau, le stockage et les machines virtuelles.



Figure 3.4. Logo OpenStack

## b. Services OpenStack

L'architecture OpenStack repose sur un ensemble de services modulaires interconnectés, chacun responsable d'un domaine fonctionnel précis. Les principaux services sont les suivants :

- **Keystone** : service d'identité et d'authentification, responsable de la gestion des utilisateurs, des projets et des jetons d'accès.
- **Nova** : service de gestion des ressources de calcul et des instances virtuelles, qui s'appuie par défaut sur l'hyperviseur KVM pour l'exécution des machines virtuelles.
- **Glance** : service de gestion du catalogue d'images de machines virtuelles, sollicité lors du démarrage d'une nouvelle instance.
- **Neutron** : service réseau qui gère la création des réseaux virtuels, des sous-réseaux et des règles de sécurité, et assure la connectivité entre les VMs ainsi qu'avec l'extérieur.
- **Cinder** : service de gestion du stockage en mode bloc, permettant l'attachement de volumes persistants aux instances.
- **Horizon** : tableau de bord web qui fournit une interface graphique pour administrer l'ensemble de l'infrastructure.

## II. Architecture globale du framework

### 1. Définition d'un framework

Un **framework** désigne un ensemble structuré de composants logiciels, d'outils et de règles d'organisation permettant de développer et d'intégrer une solution autour d'une problématique donnée. Il fournit une base architecturale cohérente, facilitant la mise en œuvre, la réutilisation des composants et l'évolution du système.

### 2. Principe de conception du framework

Notre framework repose sur un principe fondamental mis en évidence lors de l'étude de l'état de l'art : pour être fiable face à des menaces capables de subvertir les mécanismes de détection internes, l'analyse doit être conduite depuis une couche extérieure et de confiance. Dans notre architecture, la machine analysée est une **machine virtuelle Windows**, qui constitue le système invité susceptible d'héberger un éventuel rootkit. Le framework s'articule autour de quatre modules principaux, chacun ayant une responsabilité bien délimitée et communiquant

avec les autres selon un flux de données unidirectionnel : **le module d'introspection mémoire**, chargé de l'acquisition des données brutes ; **le module d'analyse mémoire**, qui interprète ces données ; **le module d'extraction de caractéristiques**, qui transforme les informations obtenues en vecteurs numériques exploitables ; et enfin **le module de détection**, qui produit la décision finale. Cette architecture modulaire facilite la maintenance, permet d'améliorer chaque composant indépendamment et offre une extensibilité naturelle vers de futurs cas d'usage.

#### a. Module d'introspection mémoire du framework

Le module d'introspection mémoire constitue le point d'entrée du processus d'analyse du framework. Son rôle est d'acquérir les données mémoire de la machine virtuelle cible à partir de dumps mémoire afin de fournir une représentation exploitable de l'état interne du système. Un **dump mémoire** est une image instantanée de la mémoire vive d'une machine virtuelle à un instant donné. Il capture l'intégralité de l'état du système en cours d'exécution : processus actifs, connexions réseau ouvertes, modules chargés dans le noyau, structures de données internes. C'est à partir de cette image que toutes les analyses ultérieures sont conduites.

Deux stratégies d'acquisition sont envisageables :

##### i. Dumps périodiques

Les dumps périodiques consistent à effectuer automatiquement des captures mémoire à intervalles réguliers afin de suivre l'évolution de l'état de la machine virtuelle. Ils permettent d'assurer une surveillance continue du système et facilitent la détection de comportements persistants. En contrepartie, cette approche génère un volume de données plus important et accroît les besoins en stockage.

##### ii. Dumps à la demande

Les dumps à la demande sont déclenchés uniquement lorsqu'un événement particulier le justifie, par exemple lors de l'exécution d'un scénario d'attaque ou à la suite de la détection d'un comportement suspect. Cette approche limite le volume de données collectées et réduit les coûts de stockage, mais peut ne pas capturer certains événements de courte durée.

Les deux stratégies présentent des avantages et des limites complémentaires, résumés dans le tableau suivant :

| Méthode           | Fonctionnement                                                              | Avantages                                                                      | Inconvénients                                                                      |
|-------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Dump périodique   | Capture automatique à intervalles réguliers (ex. toutes les 3 ou 5 minutes) | Suivi continu de l'état du système, détection possible de menaces persistantes | Coût en stockage important, risque de captures redondantes en période d'inactivité |
| Dump à la demande | Capture déclenchée manuellement ou sur événement suspect                    | Économie de ressources, ciblage précis                                         | Risque de manquer des événements éphémères entre deux déclenchements               |

Tableau 3.1. Stratégies de capture mémoire

Ces deux approches pouvant être complémentaires, le framework adopte donc une stratégie hybride combinant des captures périodiques pour assurer une surveillance continue et des captures à la demande lors de scénarios d'attaque spécifiques, conciliant ainsi détectabilité et maîtrise des ressources.

#### b. Module d'analyse mémoire du framework

Le module d'analyse mémoire est un composant central du framework. Il exploite les dumps mémoire acquis afin d'en extraire des informations structurées sur l'état interne de la machine virtuelle. Cette étape nécessite un outil capable de reconstruire les structures logiques du système d'exploitation à partir des données brutes. Pour cela trois solutions d'analyse mémoire ont été étudiées : **Volatility**, **Rekall** et **LibVMI**.

##### i. Outils d'analyse mémoire

**Volatility** [16] est un framework utilisé dans le domaine de la forensic mémoire. Il permet d'analyser des dumps mémoire et d'extraire des informations structurées telles que les processus actifs, les connexions réseau, les modules du noyau ou encore les hooks système.



Figure 3.5. Logo Volatility

**Rekall** [17] propose des fonctionnalités similaires et offre de bonnes performances. Toutefois, son développement a été officiellement interrompu et son dépôt archivé en octobre 2020, ce qui limite son évolution et sa compatibilité avec les systèmes récents.

À l'inverse, **LibVMI** [18] permet d'accéder directement à la mémoire des machines virtuelles en temps réel, sans recourir à des dumps mémoire. Bien que cette approche soit particulièrement adaptée aux architectures VMI, sa mise en œuvre nécessite une intégration étroite avec l'hyperviseur ainsi qu'une maîtrise avancée des mécanismes de virtualisation, ce qui dépasse le cadre de notre projet.

Au regard de ces éléments, **Volatility** a été retenu comme outil principal d'analyse mémoire. Son vaste écosystème de plugins (pslist, psscan, netscan, dlllist, malfind, modules, callbacks, driverirp, etc.) permet d'extraire directement les artefacts nécessaires à la détection des comportements malveillants. Par ailleurs, son adoption largement répandue dans la communauté de la forensic numérique favorise la reproductibilité et la comparabilité des résultats avec les travaux existants.

## ii. Plugins Volatility 3

Dans le cas de Windows, Volatility 3 s'appuie sur les tables de symboles (Symbol Tables en anglais) construites à partir des fichiers PDB publiés par Microsoft, qui décrivent les structures du noyau (ntoskrnl) pour chaque version du système. Ainsi, il télécharge et associe automatiquement les symboles Windows appropriés à partir de l'empreinte du noyau détectée dans le dump, ce qui simplifie la phase de configuration.

Plusieurs plugins Volatility 3 de la famille Windows ont été exploités afin d'extraire les informations nécessaires à la détection :

| Plugin Volatility 3                  | Rôle principal                                                                             |
|--------------------------------------|--------------------------------------------------------------------------------------------|
| <b>windows.modules.Modules</b>       | Énumération des modules (pilotes) chargés dans le noyau et de leurs chemins de chargement. |
| <b>windows.modscan.ModScan</b>       | Balayage mémoire des modules afin d'identifier également les modules dissimulés.           |
| <b>windows.callbacks.Callbacks</b>   | Recensement des callbacks noyau et de leurs modules propriétaires.                         |
| <b>windows.driverirp.DriverIrp</b>   | Analyse des routines d'entrée-sortie (IRP) des pilotes.                                    |
| <b>windows.devicetree.DeviceTree</b> | Analyse des attachements et relations entre périphériques                                  |

|                                      |                                                                                                                                                         |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>windows.pslist.PsList</b>         | Énumération des processus visibles via les structures officielles du noyau.                                                                             |
| <b>windows.psscan.PsScan</b>         | Balayage mémoire des processus, y compris ceux potentiellement dissimulés                                                                               |
| <b>windows.cmdline.CmdLine</b>       | Extraction des lignes de commande associées aux processus.                                                                                              |
| <b>windows.dlllist.DllList</b>       | Inventaire des bibliothèques (DLL) chargées par les processus.                                                                                          |
| <b>windows.ldrmodules.LdrModules</b> | Comparaison des trois listes de modules chargés maintenues par le chargeur Windows afin d'identifier les incohérences révélatrices d'une dissimulation. |
| <b>windows.driverscan.DriverScan</b> | Balayage mémoire des objets pilotes, y compris ceux dont les structures de métadonnées ont été altérées.                                                |
| <b>windows.thrdscan.ThrdScan</b>     | Balayage mémoire des threads, permettant d'identifier ceux ne pouvant être rattachés à un module connu.                                                 |
| <b>windows.ssdt.SSDT</b>             | Énumération des entrées de la table des appels système et des modules vers lesquels elles pointent.                                                     |
| <b>windows.timers.Timers</b>         | Recensement des minuteurs noyau et des modules auxquels ils sont associés.                                                                              |

Tableau 3.2. Plugins Volatility 3 utilisés pour l'extraction des caractéristiques

Ces plugins constituent la principale source d'information exploitée par le module d'extraction. Leurs résultats sont utilisés pour calculer les caractéristiques comportementales du framework, certaines reposant sur l'analyse d'un plugin unique, tandis que d'autres nécessitent le croisement d'informations provenant de plusieurs plugins afin d'obtenir une représentation plus fidèle de l'état réel de la machine virtuelle.

### c. Module d'extraction de caractéristiques du framework

Une caractéristique (feature) est une information extraite des données représentant l'état ou le comportement du système analysé. Les informations issues de l'analyse mémoire réalisée par Volatility sont nombreuses mais hétérogènes. Avant de pouvoir être exploitées par les algorithmes d'apprentissage automatique, elles doivent être sélectionnées, nettoyées et transformées en représentations numériques structurées. Le module d'extraction de caractéristiques assure cette transformation en construisant un vecteur numérique décrivant l'état de la machine virtuelle. Il constitue ainsi l'interface entre l'analyse mémoire forensique et le module de détection.

#### i. Caractéristiques extraites

Le pouvoir de détection du framework repose entièrement sur la pertinence des caractéristiques extraites de la mémoire. Une caractéristique mal choisie peut soit rester silencieuse face à une menace, soit produire des fausses alertes sur un système sain.



La sélection a donc fait l'objet d'une démarche progressive : partant d'un ensemble de caractéristiques hérité de travaux antérieurs, leur pertinence réelle sur un système Windows moderne a été évaluée, conduisant à conserver, écarter ou compléter chaque indicateur.

- **Les caractéristiques historiques (héritées de l'approche TKRD)**

Un premier ensemble de caractéristiques a été construit à partir d'indicateurs largement utilisés dans la littérature sur la détection de rootkits par analyse mémoire, notamment dans l'approche TKRD [11].

Cette approche repose sur **huit caractéristiques** décrivant différents artefacts du noyau. Afin d'élargir la couverture des comportements susceptibles d'être observés en mémoire, quatre indicateurs supplémentaires, **total\_modules**, **total\_threads**, **total\_drivers** et **total\_callback**, ont été ajoutés. L'ensemble de ces caractéristiques vise à mettre en évidence différents artefacts susceptibles de révéler la présence d'un rootkit au niveau du noyau.

#### Indicateurs de dissimulation d'objets noyau

| Caractéristique       | Description                                                              |
|-----------------------|--------------------------------------------------------------------------|
| <b>hidden_modules</b> | Modules noyau présents en mémoire mais absents des listes officielles.   |
| <b>orphan_threads</b> | Threads ne pouvant être rattachés à aucun module connu.                  |
| <b>drivers_hidden</b> | Pilotes détectés par scan mémoire mais absents des structures attendues. |
| <b>ssdt_hooks</b>     | Modifications suspectes de la table des appels système (SSDT).           |

Tableau 3.3. Indicateurs de dissimulation d'objets noyau

#### Indicateurs de détournement et d'interception

| Caractéristique                | Description                                                  |
|--------------------------------|--------------------------------------------------------------|
| <b>irp_hooks</b>               | Suspicion de détournement des routines IRP.                  |
| <b>device_attachments_susp</b> | Attachements suspects dans l'arborescence des périphériques. |
| <b>callbacks_suspects</b>      | Callbacks associés à des modules suspects.                   |
| <b>timers_suspects</b>         | Minuteurs noyau associés à des objets suspects.              |

Tableau 3.4. Indicateurs de détournement et d'interception

#### Indicateurs contextuels

| Caractéristique        | Description                            |
|------------------------|----------------------------------------|
| <b>total_modules</b>   | Nombre total de modules noyau.         |
| <b>total_threads</b>   | Nombre total de threads.               |
| <b>total_drivers</b>   | Nombre total de pilotes chargés.       |
| <b>total_callbacks</b> | Nombre total de callbacks enregistrés. |

Tableau 3.5. Indicateurs contextuels

- **Les caractéristiques détection de la couche noyau**

Un second ensemble de six caractéristiques a été défini, spécifiquement adapté aux mécanismes employés par les rootkits noyau contemporains sous Windows x64. Leur principe directeur est de ne pas chercher une technique de furtivité particulière, mais de s'appuyer sur un invariant structurel : un pilote légitime est chargé depuis un emplacement système contrôlé et déclaré de manière standard, tandis qu'un rootkit s'en écarte nécessairement à un moment de son cycle de vie. La première caractéristique, `drivers_outside_system32`, dénombre les modules noyau dont le chemin de chargement se situe en dehors des répertoires système légitimes.

Les trois caractéristiques suivantes `irp_hooks_suspicious`, `callbacks_suspicious`, `device_attach_suspicious` reposent sur le même principe d'attribution : elles ne comptabilisent pas l'ensemble des callbacks, routines IRP ou attachements de périphériques trop nombreux et largement légitimes, mais uniquement ceux attribués à un module identifié comme non-système. La cinquième, `drivers_dos_device_path`, capte les pilotes chargés selon une notation non standard. Un pilote installé par le gestionnaire de services référence son image via un chemin de forme `\SystemRoot...`, tandis qu'un chargement manuel produit une notation de périphérique DOS de forme `??\C:....`. Cette caractéristique dénombre les modules adoptant cette dernière forme, indice de chargement non conventionnel indépendant de l'emplacement du fichier. Et la sixième, `hidden_processes_active`, vise la manipulation directe des objets du noyau (DKOM) : un processus présent au balayage mémoire (`windows.psscan`) mais absent de la liste officielle (`windows.pslist`), et dont les threads restent actifs, est considéré comme potentiellement masqué.

| Caractéristique                 | Description synthétique           |
|---------------------------------|-----------------------------------|
| <b>drivers_outside_system32</b> | Pilotes hors répertoires système. |

|                                 |                                                                               |
|---------------------------------|-------------------------------------------------------------------------------|
| <b>irp_hooks_suspicious</b>     | Routines IRP suspectes.                                                       |
| <b>callbacks_suspicious</b>     | Callbacks suspects.                                                           |
| <b>device_attach_suspicious</b> | Attachements suspects.                                                        |
| <b>drivers_dos_device_path</b>  | Chemins DOS non standards.                                                    |
| <b>hidden_processes_active</b>  | Processus actifs détectés en mémoire mais absents des structures officielles. |

Tableau 3.6. Caractéristiques noyau retenues

- **Les caractéristiques de la couche espace utilisateur (userland)**

En complément de la couche de détection noyau, une seconde couche est destinée à l'analyse des artefacts observables en espace utilisateur. Elle repose sur un ensemble de caractéristiques décrivant des comportements susceptibles de révéler des techniques d'injection, de dissimulation ou de persistance observables en espace utilisateur.

Dans cette optique, quelques caractéristiques ont été identifiées afin de constituer le moteur d'apprentissage automatique de cette couche.

| <b>Caractéristique</b>         | <b>Description synthétique</b>                                                                              |
|--------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>procs_suspicious_path</b>   | Processus exécutés depuis un emplacement non standard.                                                      |
| <b>dlls_suspicious_path</b>    | DLL chargées depuis un emplacement non standard.                                                            |
| <b>procs_no_cmdline</b>        | Processus ne possédant pas de ligne de commande.                                                            |
| <b>ldrmodules_inconsistent</b> | Incohérences entre les listes de modules du chargeur Windows, pouvant révéler des DLL cachées ou injectées. |

Tableau 3.7. Caractéristiques de détection userland

Bien que ces caractéristiques aient été définies dans le contexte de la détection de rootkits (noyau et userland), elles ne leur sont pas exclusivement propres. Plusieurs d'entre elles correspondent à des comportements également observés chez d'autres catégories de malwares avancés, notamment ceux recourant à des techniques d'injection de code, de persistance ou de chargement non conventionnel. Le framework conserve ainsi une capacité potentielle de détection au-delà des seuls rootkits, sous réserve que ces comportements se traduisent par des artefacts mémoire compatibles avec les caractéristiques analysées. Cette liste de

caractéristiques n'est toutefois pas exhaustive et pourra être enrichie à mesure que le framework gagnera en maturité.

## ii. Représentation vectorielle pour le Machine Learning

Une fois les caractéristiques extraites, elles doivent être transformées en un format exploitable par les algorithmes de Machine Learning (ML). Cette étape consiste à convertir les informations qualitatives en **vecteurs numériques**. Plusieurs approches peuvent être utilisées :

- **Approche manuelle (feature engineering)** : Cette approche consiste à sélectionner manuellement les caractéristiques considérées comme pertinentes pour l'analyse (ex : nombre de processus, présence d'anomalies) ;
- **Approche automatique** : Dans cette approche, les caractéristiques sont extraites automatiquement à partir des données grâce à des techniques avancées d'analyse ou d'apprentissage. Le système cherche ainsi à identifier des relations ou des comportements complexes difficiles à détecter manuellement ;
- **Approche hybride** : Cette approche combine les deux méthodes précédentes. Elle utilise à la fois des caractéristiques définies manuellement et des caractéristiques découvertes automatiquement afin d'obtenir une représentation plus complète des données.

Pour notre travail, une approche principalement manuelle enrichie a été retenue. Les variables sont choisies et construites en fonction de leur signification comportementale directe, garantissant ainsi que chaque dimension du vecteur reste interprétable et rattachable à un comportement système concret, ce qui facilitera l'analyse des résultats produits par le module de détection.

## d. Module de détection

Le module de détection est le cœur décisionnel du framework. Il reçoit les vecteurs de caractéristiques produits par le module précédent et produit une classification : le comportement observé est-il normal ou malveillant ? Contrairement aux approches basées sur des signatures, qui ne peuvent identifier que des menaces déjà répertoriées, ce module est conçu pour détecter des comportements anormaux inconnus.

## i. Approche Machine Learning

Trois paradigmes sont pertinents dans le contexte de la détection de malwares.

- **L'apprentissage supervisé** entraîne un modèle sur des données étiquetées (systèmes sains vs systèmes infectés). Il offre d'excellentes performances de classification lorsque le jeu de données est représentatif et bien équilibré. Sa principale limite est sa dépendance à la qualité et à la complétude des données annotées : **un modèle supervisé peinera à reconnaître une variante de malware absente de son jeu d'entraînement.**
- **L'apprentissage non supervisé** ne s'appuie sur aucune étiquette. Il vise à identifier des comportements qui s'écartent significativement du fonctionnement normal. Cette approche est particulièrement adaptée à la détection de menaces inconnues (zero-day), mais génère davantage de faux positifs et est plus difficile à calibrer.
- **Les approches hybrides** combinent les deux paradigmes : un modèle supervisé traite les menaces connues avec précision, tandis qu'un détecteur d'anomalies signale les comportements inhabituels non répertoriés. Cette combinaison offre la meilleure couverture théorique, au prix d'une **complexité de conception et d'entraînement plus élevée.**

| Approche                            | Avantages                                                    | Limites                                                                                                 | Pertinence pour le projet                                                                 |
|-------------------------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Supervisée                          | Haute précision, faible taux de faux positifs.               | Requiert un dataset étiqueté représentatif, sensible aux familles de menaces non vues à l'entraînement. | Dépendance à un dataset étiqueté et nécessité de réentraînement à chaque nouvelle menace. |
| Non supervisée                      | Détection de menaces inconnues, aucun étiquetage nécessaire. | Plus de faux positifs, calibration délicate.                                                            | Adaptée à la détection de menaces zero-day.                                               |
| Hybride (supervisé + non supervisé) | Couvre menaces connues et inconnues.                         | Nécessite davantage de données et de ressources.                                                        | Complexité d'orchestration de deux modèles (supervisé et non supervisé).                  |

Tableau 3.8. Tableau comparatif approche Machine Learning

La stratégie adoptée repose principalement sur l'apprentissage non supervisé, choisi pour sa capacité à détecter des menaces inédites sans nécessiter de données préalablement étiquetées, propriété essentielle face à des rootkits et malwares dont les variantes évoluent continuellement.

Le choix des modèles retenus repose sur plusieurs critères directement liés aux objectifs du projet. En premier lieu, les modèles doivent pouvoir fonctionner **en mode one-class et non supervisé**, c'est-à-dire d'apprendre la structure du comportement majoritaire à partir des caractéristiques des observations sans utiliser **leurs étiquettes lors de l'entraînement**. Ils doivent également être capables **d'identifier des comportements anormaux sans connaissance préalable des attaques**, afin de détecter des variantes inédites ou des menaces de type **zero-day** [19]. Par ailleurs, les modèles sélectionnés doivent être adaptés à des vecteurs de caractéristiques numériques de faible dimension, correspondant aux données produites par le module d'extraction. Enfin, ils doivent permettre le calcul d'un seuil de décision contrôlant le **taux de faux positifs**, condition indispensable pour garantir l'exploitabilité opérationnelle du détecteur.

Sur cette base, plusieurs modèles d'apprentissage automatique non supervisé ont été retenus et feront l'objet d'une évaluation expérimentale afin d'identifier celui offrant le meilleur compromis entre capacité de détection, robustesse et maîtrise des faux positifs.

- **Isolation Forest**

Isolation Forest (IF) est un algorithme de détection d'anomalies fondé sur le principe que les observations atypiques sont plus faciles à isoler que les observations normales. Il construit un ensemble d'arbres de partition aléatoires dans lesquels les observations anormales sont généralement isolées après un faible nombre de divisions. Le modèle est entraîné à partir des vecteurs de caractéristiques extraits d'un corpus non trié, dont aucune observation n'est étiquetée. Cet algorithme a été retenu pour sa simplicité de mise en œuvre, son faible coût de calcul et sa capacité à détecter des comportements atypiques sans connaissance préalable des menaces.

- **Local Outlier Factor**

Local Outlier Factor (LOF) est un algorithme de détection d'anomalies reposant sur l'analyse de la densité locale des observations. Son principe consiste à comparer la densité d'une observation à celle de ses plus proches voisins : une observation située dans une région de densité significativement plus faible est considérée comme anormale. Lors de l'entraînement, il apprend la distribution des vecteurs de caractéristiques issus du corpus non trié afin d'estimer les régions correspondant au comportement normal. LOF a été retenu pour sa capacité à mettre en évidence des anomalies locales susceptibles d'échapper à une analyse globale des données.

- **One-Class Support Vector Machine**

One-Class Support Vector Machine (One-Class SVM) est un algorithme de détection d'anomalies qui apprend une frontière délimitant le domaine occupé par les observations correspondant au comportement normal. Toute observation située en dehors de cette frontière est considérée comme potentiellement anormale. Lors de l'entraînement, cette frontière est construite à partir des vecteurs de caractéristiques extraits d'un corpus de dumps mémoire non étiquetés, dont on suppose seulement qu'une fraction minoritaire correspond à des systèmes compromis. Cet algorithme a été retenu pour sa capacité à modéliser efficacement un comportement de référence à partir d'observations saines uniquement, ce qui le rend particulièrement adapté à la détection d'anomalies.

- ii. Pipeline de traitement

Le fonctionnement du module de détection suit un pipeline structuré en cinq étapes, depuis la réception des caractéristiques extraites par le module d'analyse forensique jusqu'à la production du verdict final.

- **Étape 1 : Chargement des caractéristiques**

Le vecteur de caractéristiques produit par le module d'analyse forensique est chargé et vérifié avant toute opération de détection.

- **Étape 2 : Prétraitement**

Les caractéristiques extraites sont converties sous la forme attendue par les modèles de détection. Cette étape comprend notamment la sélection des variables retenues lors de l'entraînement, leur organisation dans un vecteur de caractéristiques cohérent ainsi que, lorsque nécessaire, leur normalisation afin de garantir une échelle comparable entre les différentes variables.

- **Étape 3 : Organisation des caractéristiques par couches d'observation**

Les caractéristiques sont organisées en deux ensembles complémentaires correspondant aux deux niveaux d'observation du framework. La première couche regroupe les caractéristiques décrivant les artefacts du noyau du système d'exploitation observé (Ring 0), tandis que la seconde rassemble les caractéristiques représentant les manifestations observables en espace utilisateur (Ring 3). Cette organisation facilite l'interprétation des résultats sans modifier le

processus de décision et est purement destinée à l'interprétation des résultats. Le vecteur transmis au modèle demeure unique et l'ensemble des caractéristiques y est traité de manière indifférenciée.

- **Étape 4 : Calcul du score d'anomalie**

Le vecteur prétraité est soumis au moteur d'apprentissage automatique, qui produit un score d'anomalie mesurant l'écart entre l'état observé de la machine virtuelle et le domaine de normalité appris lors de l'entraînement.

Ce moteur est entraîné en régime non supervisé : le corpus d'apprentissage lui a été transmis sans aucune étiquette, et aucune information sur la nature des observations n'est intervenue ni dans l'apprentissage, ni dans la calibration du seuil.

La convention de score est uniforme :

$$S = - \text{score\_samples}(X)$$

Un score élevé traduit une observation plus atypique. Cette convention rend les seuils comparables entre algorithmes, dont les sorties natives diffèrent en signe et en échelle.

- **Étape 5 : Production du verdict**

Le score d'anomalie est comparé au seuil de décision déterminé lors de la phase d'entraînement et exporté avec le modèle :

$$\text{Verdict} = \text{Infectée si } S > T, \text{ Saine sinon}$$

Où S désigne le score d'anomalie et T le seuil de décision.

Le calcul de T ne fait intervenir aucune étiquette. Il repose sur l'hypothèse de **contamination**  $v$  du corpus d'entraînement, c'est-à-dire la proportion supposée d'observations anormales qu'il contient. Soit  $S_{\text{train}} = \{s_1, s_2, \dots, s_N\}$  l'ensemble des scores d'anomalie produits par le modèle sur le corpus d'entraînement. Le seuil retenu est le quantile empirique d'ordre  $(1 - v)$  de cette distribution :

$$T = Q_{1-v}(S_{\text{train}})$$

Le seuil est ainsi positionné de sorte qu'une proportion  $v$  des observations d'entraînement le dépasse.



Afin d'apporter une aide à l'interprétation du verdict, le framework calcule **un indice de confiance non probabiliste**. Cet indice indique à quel point le score d'une observation s'éloigne du seuil de décision, en tenant compte de la dispersion des scores observés sur le corpus d'entraînement.

Soit MAD l'écart absolu médian des scores d'entraînement. Pour un score observé  $S$ , on définit l'écart normalisé :

$$Z = \frac{(S - T)}{MAD}$$

L'indice de confiance s'écrit alors :

$$C_{\text{infectée}} = \frac{1}{(1 + e^{-z})} \text{ et } C_{\text{saine}} = 1 - C_{\text{infectée}}$$

$C = C_{\text{infectée}}$  si le verdict est Infectée,  $C_{\text{saine}}$  si le verdict est Saine.

Notre pipeline Figure 3.6 assure une transition cohérente entre les données mémoire brutes, l'analyse forensique et le processus de détection. La combinaison des caractéristiques issues des couches noyau et utilisateur permet au moteur d'apprentissage d'exploiter une représentation plus complète de l'état de la machine virtuelle, tandis que la séparation logique de ces couches facilite l'interprétation des résultats et l'identification des artefacts ayant le plus contribué au verdict final.

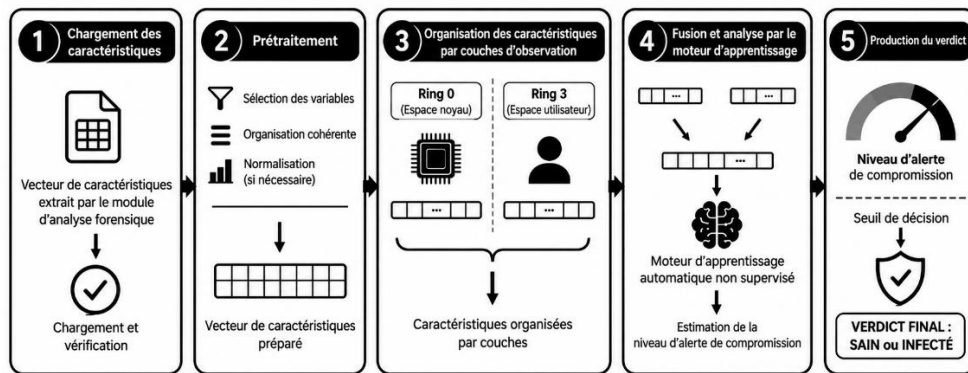


Figure 3.6. Pipeline du module de détection

### 3. Flux de données entre les modules

Les quatre modules décrits précédemment s'inscrivent dans une chaîne de traitement linéaire et unidirectionnelle, où chaque module consomme la sortie du précédent pour produire une représentation plus abstraite et exploitable.

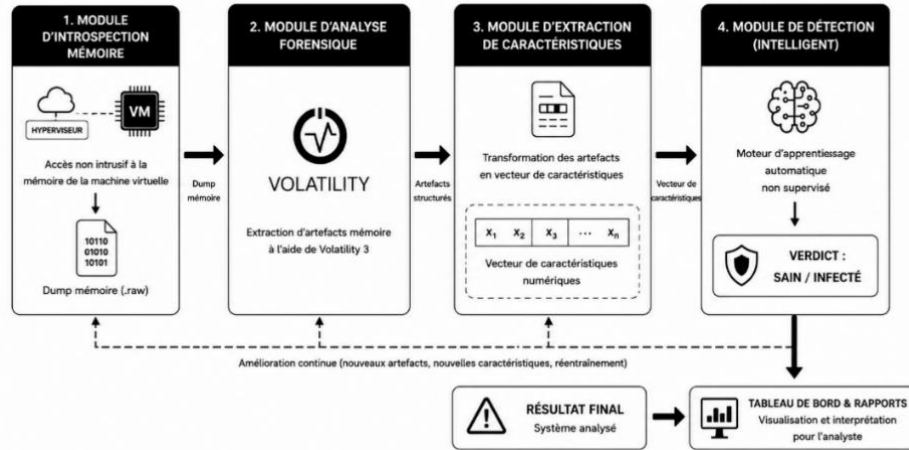


Figure 3.7. Schéma représentatif du flux de données entre les modules du framework

Ce flux présente plusieurs propriétés importantes pour la robustesse et l'évolutivité du framework. D'abord, chaque étape produit une sortie dans un format standardisé, ce qui rend les modules interchangeables : on pourrait par exemple remplacer Volatility par un autre outil d'analyse mémoire sans modifier le reste de la chaîne, à condition de respecter le format de sortie attendu.

Ensuite, la linéarité du flux simplifie le débogage : en cas de résultat incorrect, il est possible d'inspecter la sortie de chaque module individuellement pour identifier l'étape défaillante. Par ailleurs, ce découplage entre les modules permet d'envisager à terme une exécution distribuée ou parallèle de certaines étapes sans remettre en cause l'architecture globale.

### Conclusion

Ce chapitre a présenté les choix technologiques et l'architecture modulaire du framework, depuis l'introspection mémoire jusqu'au module de détection. Ces fondations seront mises en œuvre dans le chapitre suivant, qui détaille l'implémentation de chacun des composants du framework.

## Chapitre 4 : Implémentation des composants du framework

### Introduction

Ce chapitre détaille la réalisation des composants choisis pour le framework présenté au chapitre précédent, depuis la mise en place de l'environnement de travail jusqu'à l'intégration des différents modules et de la chaîne de traitement associée.

### I. Environnement de développement et de déploiement

#### 1. Environnement matériel

Notre environnement matériel repose sur une machine physique principale qui héberge l'ensemble de la plateforme cloud et des outils d'analyse. Ses caractéristiques sont résumées dans le tableau suivant :

| Composant    | Caractéristique                                                                  |
|--------------|----------------------------------------------------------------------------------|
| Marque       | LENOVO                                                                           |
| Processeur   | Intel x86-64 avec support des extensions de virtualisation Intel VT-x (flag vmx) |
| Mémoire vive | 7,4 Go                                                                           |
| Système hôte | Ubuntu 24.04 LTS                                                                 |

Tableau 4.1. Caractéristiques de la machine hôte (nœud de supervision)

Cette configuration mono-nœud, aux ressources limitées, héberge à la fois les services cloud, l'hyperviseur, les outils d'introspection et les modules du framework. Bien qu'elle ne reproduise pas la répartition multi-serveurs d'un cloud de production, elle constitue un compromis pertinent entre réalisme expérimental et contraintes matérielles, et permet de valider l'ensemble de la chaîne de traitement dans un environnement reproductible et maîtrisé.

#### 2. Environnement logiciel

La mise en œuvre du framework nécessite un environnement logiciel capable de supporter la virtualisation, l'orchestration cloud, l'analyse mémoire ainsi que les mécanismes de détection basés sur le Machine Learning. Cette section présente les différentes étapes de mise en place de l'infrastructure logicielle utilisée lors de la réalisation du projet.

#### a. Mise en place de KVM/QEMU

L'environnement de virtualisation repose sur l'utilisation du couple KVM/QEMU installé sur le nœud de supervision fonctionnant sous Ubuntu 24.04.

La mise en place de cet environnement a débuté par la vérification du support matériel de virtualisation sous Ubuntu 24.04 (voir Figure 4.1), afin de s'assurer que le processeur prenait bien en charge les extensions **Intel VT-x** nécessaires au fonctionnement de KVM, avant l'installation des différents paquets liés à l'environnement de virtualisation.

```
line@line-ubuntu:~$ grep -E 'vmx|svm' /proc/cpuinfo
flags       : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse
sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc art arch_perfmon pebs bts rep_good nopl xtopology nonstop_
tsc cpuid aperfperf tsc_known_freq pni pclmulqdq dtes64 monitor ds_cpl vmx est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid
sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault ep
b ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow flexpriority ept vpid ept_ad fsgsbase tsc_adjust sgx bmi1 avx2 smep bmi
2 erms invpcid avx512f avx512dq rdseed adx smap avx512ifma clflushopt intel_pt avx512cd sha_ni avx512bw avx512vl xsaveo
pt xsavec xgetbv1 xsave split_lock_detect dtherm ida arat pln pts hwp hwp_notify hwp_act_window hwp_epp hwp_pkg_req vn
mi avx512vbmi umip pku ospke avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni avx512_bitalg avx512_vpopcntdq rdpid sgx_lc
fsrm md_clear flush_l1d arch_capabilities
vmx flags    : vmmi preempt_tmr posted_intr invvpid ept_x_only ept_ad ept_1gb flexpriority apicv tsc_offset vtp
r mtf vapid ept vpid unrestricted_guest vapid_reg vid ple pml ept_violation_ve ept_mode_based_exec tsc_scaling
flags       : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse
```

Figure 4.1. Vérification du support matériel de virtualisation

L'analyse du fichier `/proc/cpuinfo` sous Linux a permis d'identifier la présence du flag `vmx`, indiquant que le processeur supporte la technologie de virtualisation **Intel VT-x**. Cette information confirme la compatibilité matérielle du système avec les solutions de virtualisation.

L'installation de KVM a ensuite été réalisée à l'aide des paquets `qemu-kvm`, `libvirt-daemon-system`, `libvirt-clients` et `virt-manager`. Une fois l'installation terminée, les services `libvirtd` ont été activés afin d'assurer le fonctionnement et la gestion des machines virtuelles sur le système hôte.

QEMU est utilisé conjointement avec KVM afin d'assurer l'émulation des composants matériels virtuels, notamment les interfaces réseau, les périphériques de stockage et les ressources d'entrée/sortie nécessaires au fonctionnement des machines virtuelles. L'intégration entre KVM/QEMU et `libvirt` constitue ainsi la base de l'environnement de virtualisation sur lequel repose l'ensemble du framework.

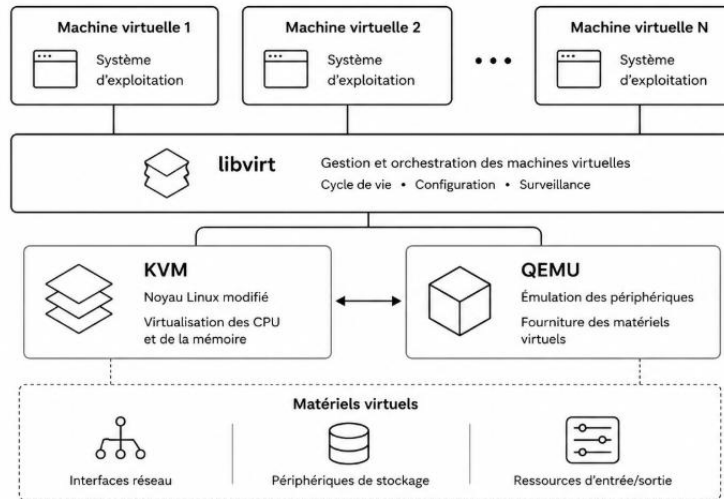


Figure 4.2. Environnement de virtualisation KVM/QEMU

#### b. Déploiement de DevStack/OpenStack

Après la mise en place de l'environnement de virtualisation, la plateforme cloud OpenStack a été déployée à l'aide de DevStack.

Cette infrastructure agit comme une forme de cloud privé local mis en place pour les expérimentations, permettant de gérer les machines virtuelles, le réseau et les ressources de virtualisation au sein du framework. Le déploiement a été réalisé à partir des scripts officiels de DevStack après le téléchargement du dépôt correspondant.

La configuration principale du déploiement a été réalisée à travers le fichier `local.conf` (voir Figure 4.3), dans lequel ont été définis les paramètres essentiels de l'infrastructure cloud, notamment les mots de passe des différents services, l'adresse réseau utilisée, le type d'hyperviseur retenu ainsi que les composants OpenStack à activer.

```
GNU nano 7.2 local.conf
[[local|localrc]]
ADMIN_PASSWORD=
DATABASE_PASSWORD=
RABBIT_PASSWORD=
SERVICE_PASSWORD=

HOST_IP=127.0.0.1
```

Figure 4.3. Configuration de déploiement DevStack

Une fois cette configuration terminée, le script `stack.sh` a été exécuté afin d'initialiser automatiquement les principaux services OpenStack.

L'achèvement de cette phase d'initialisation a permis l'accès à l'interface web Horizon d'OpenStack depuis un navigateur (voir Figure 4.4).

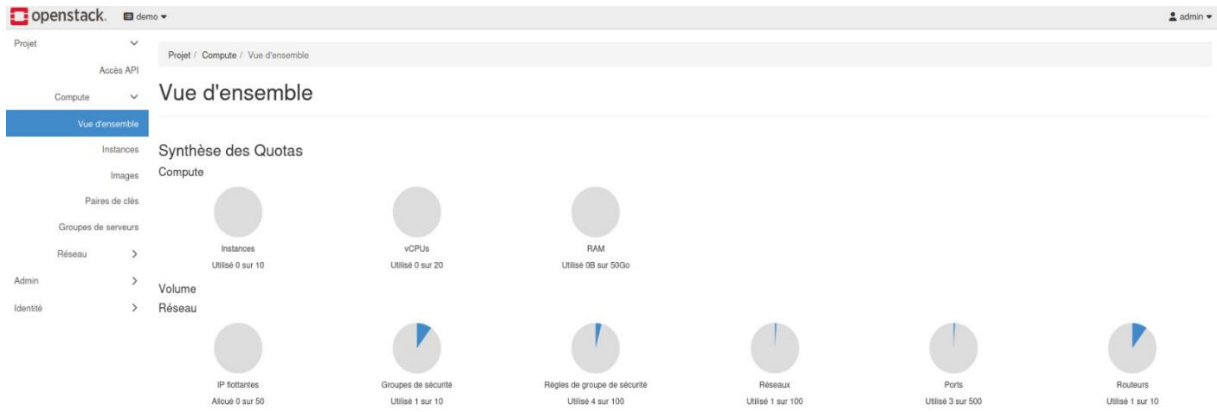


Figure 4.4. Dashboard OpenStack

### c. Outils et dépendances du framework

Le développement du framework repose principalement sur le langage Python, retenu pour la richesse de son écosystème en matière d'analyse de données et d'apprentissage automatique, ainsi que pour sa capacité à automatiser l'enchaînement des différentes étapes de traitement. Python assure l'implémentation de la majorité des modules du système : l'orchestration des captures mémoire, l'analyse des dumps, l'extraction des caractéristiques et le pipeline de détection. Des scripts Bash complètent cet ensemble pour certaines opérations système et tâches d'automatisation propres à l'environnement Linux, telles que la gestion des services ou l'enchaînement de commandes d'administration.



Figure 4.5. Logo Python et Bash

### 3. Environnement d'expérimentation

Les expérimentations ont été réalisées sur une infrastructure reposant sur OpenStack et l'hyperviseur KVM/QEMU. Une machine virtuelle **Windows 10 x64 Enterprise LTSC** a été utilisée comme système cible pour l'exécution des scénarios et le chargement des rootkits étudiés (voir Figure 4.6). Cette version de Windows a été retenue en raison de sa stabilité, de sa



large utilisation dans les environnements professionnels et de la faible variabilité de son environnement logiciel.

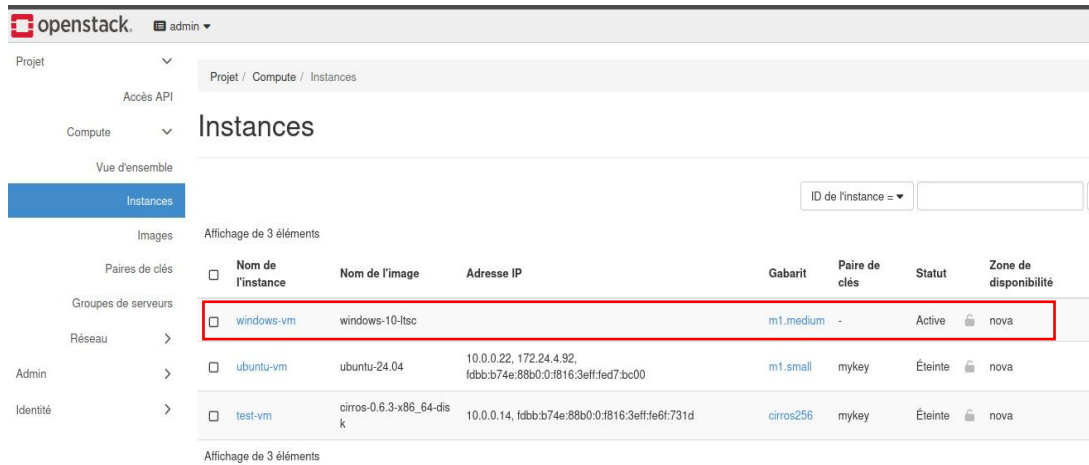


Figure 4.6. VM Windows active sur OpenStack

## II. Implémentation des modules du framework

### 1. Module d'introspection mémoire

La collecte des données mémoire repose sur l'utilisation de l'outil `virsh`, interface en ligne de commande fournie par `libvirt` permettant d'administrer les machines virtuelles gérées par KVM. Le module de capture a été développé sous la forme d'un script Python nommé `capture_windows.py`, qui constitue le composant principal du module d'introspection.

Ce script encapsule la logique d'acquisition au sein d'une classe `VMIWindowsCapture`, chargée d'identifier l'instance `libvirt` correspondant à la machine virtuelle cible puis de générer automatiquement le dump mémoire correspondant. En interne, la capture s'appuie sur la commande `virsh dump` : `sudo virsh dump --memory-only --live <instance> dump.raw`. (Voir Figure 4.7). Dans cette commande, `<instance>` représente l'identifiant de l'instance virtuelle gérée par `libvirt` (de la forme `instance-0000000X`), tandis que `dump.raw` correspond au fichier contenant l'image mémoire brute générée. L'option `--memory-only` restreint la capture au seul contenu de la mémoire vive, tandis que l'option `--live` autorise la capture sans interrompre l'exécution du système invité, préservant ainsi son état de fonctionnement durant la collecte. Ces dumps constituent la principale source de données utilisée par les modules d'analyse et de détection du framework.

```
(cloudguard-venv) line@line-ubuntu:~$ sudo su - stack
[sudo] Mot de passe de line :
stack@line-ubuntu:~$ INSTANCE=$(sudo virsh list --all --name | grep instance- | head -1)
echo "Instance : $INSTANCE"
Instance : instance-00000003
stack@line-ubuntu:~$ sudo virsh dump --memory-only --live $INSTANCE /home/line/cloudguard/dumps/clean.raw

Domain 'instance-00000003' dumped to /home/line/cloudguard/dumps/clean.raw

stack@line-ubuntu:~$ sudo chmod 644 /home/line/cloudguard/dumps/clean.raw
stack@line-ubuntu:~$ ls -lh /home/line/cloudguard/dumps/clean.raw
ls: impossible d'accéder à '/home/line/cloudguard/dumps/clean.raw': Permission non accordée
stack@line-ubuntu:~$ sudo ls -lh /home/line/cloudguard/dumps/clean.raw
-rw-r--r-- 1 root root 2,1G mai 11 13:54 /home/line/cloudguard/dumps/clean.raw
```

Figure 4.7. Présentation d'un cas de capture mémoire

## 2. Module d'analyse mémoire forensique

Une fois le dump mémoire acquis, il doit être interprété afin d'en extraire des informations structurées sur l'état du système. Cette étape repose sur Volatility 3, framework spécialisé dans l'analyse forensique de la mémoire vive.

L'installation de Volatility 3 a été réalisée dans un environnement Python dédié (environnement virtuel) afin d'isoler les dépendances du projet.

Après récupération du framework volatility, les bibliothèques requises ont été installées à l'aide du gestionnaire de paquets pip.

```
line@line-ubuntu:~$ source ~/vol3-venv/bin/activate
bash: /home/line/vol3-venv/bin/activate: Aucun fichier ou dossier de ce nom
line@line-ubuntu:~$ python3 -m venv ~/vol3-venv
line@line-ubuntu:~$ source ~/vol3-venv/bin/activate
(vol3-venv) line@line-ubuntu:~$ pip install volatility3 yara-python
```

Figure 4.8. Installation de Volatility

Pour chaque dump mémoire acquis, le module d'analyse invoque automatiquement Volatility 3 afin d'effectuer l'analyse forensique de la mémoire. Les plugins préalablement sélectionnés sont exécutés de manière séquentielle, chacun étant chargé d'extraire un type spécifique d'information, tel que les processus, les pilotes, les modules, les callbacks ou les connexions réseau. Les résultats produits par Volatility 3 sont ensuite enregistrés sous forme de fichiers texte dans un répertoire de travail. Le module automatise l'exécution des plugins et la gestion des sorties générées, garantissant une analyse reproductible et homogène pour chaque dump mémoire.



### 3. Module d'extraction de caractéristiques

Les caractéristiques sont calculées par un script `feature_extractor_windows.py` à partir des sorties des plugins Volatility 3.

Les différentes sorties générées par Volatility 3 sont d'abord chargées et analysées. Afin de garantir la cohérence des résultats, les noms des modules sont normalisés (suppression de l'extension, conversion en minuscules, etc.) avant d'associer à chacun les callbacks, routines IRP et attachements correspondants. Cette étape permet de corréler des informations provenant de plusieurs plugins, qui ne désignent pas toujours un même module de manière identique.

Le module calcule ensuite les différentes caractéristiques décrivant l'état du système, notamment à partir des artefacts du noyau et de l'espace utilisateur. Les valeurs obtenues sont regroupées dans un vecteur de caractéristiques unique, qui représente l'état de la machine virtuelle au moment de la capture mémoire. Ce vecteur est finalement enregistré au format CSV afin d'être directement exploité par le module de détection.

### 4. Module de détection

Le module de détection reçoit en entrée le vecteur de caractéristiques généré par le module d'extraction et détermine si l'état observé de la machine virtuelle correspond à un comportement normal ou à un comportement potentiellement malveillant. À partir du vecteur de caractéristiques, le module applique les mêmes opérations de prétraitement que celles utilisées lors de la phase d'entraînement (sélection des caractéristiques et normalisation), afin de garantir la cohérence entre les données d'apprentissage et les nouvelles observations. Le vecteur est ensuite soumis au modèle d'apprentissage automatique qui sera déterminé dans la section suivante (voir Tableau 5.10) préalablement entraîné. Celui-ci calcule un **score d'anomalie**, traduisant le degré d'écart entre l'observation analysée et le comportement normal appris lors de l'entraînement. Ce score est comparé au **seuil de décision** exporté avec le modèle afin de produire le verdict final du framework. En complément du verdict (Saine ou Infectée), le module calcule un **indice de confiance** permettant d'estimer le niveau de certitude associé à la décision. L'ensemble des résultats est ensuite présenté à l'utilisateur sous une forme directement exploitable.

### 5. Scheduler : surveillance continue

Afin d'automatiser la surveillance des machines virtuelles, un composant d'orchestration nommé `realtime_scheduler_windows.py` a été développé. Contrairement au script de capture, dont le rôle se limite à la génération ponctuelle d'un dump, le scheduler assure l'orchestration

complète et répétée du processus de surveillance. À intervalle régulier, il déclenche une capture mémoire en s'appuyant sur la classe VMIWindowsCapture, applique le module d'extraction de caractéristiques sur le dump obtenu, soumet le vecteur résultant au module de détection, puis journalise le verdict. Il assure également le suivi de l'état du système entre deux analyses, ce qui lui permet de signaler les transitions significatives, l'apparition d'une nouvelle menace (passage d'un état sain à un état compromis) comme sa disparition.

Le lancement du scheduler s'effectue par une commande de la forme : `python3 realtime_scheduler_windows.py --model-dir models_rootkit --interval 2` où l'option `--interval` fixe la période entre deux analyses et `--model-dir` désigne le répertoire des modèles de détection à utiliser. Le scheduler constitue ainsi le composant assurant l'automatisation globale du framework et la continuité de la surveillance. Un extrait de la pile de résultats ainsi journalisée est présenté en annexe (voir Annexe, Figure A.3)

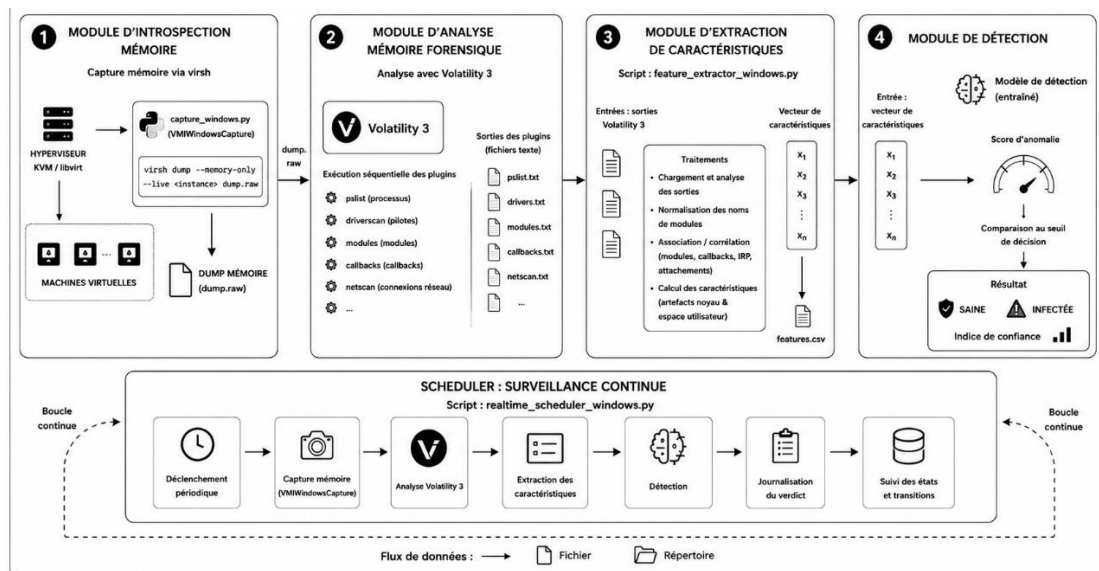


Figure 4.9. Architecture du fonctionnement du framework de détection

## Conclusion

Ce chapitre a détaillé l'implémentation des différents composants du framework ainsi que leur orchestration au sein d'un pipeline automatisé. Le chapitre suivant évalue expérimentalement les modèles de détection construits sur ces composants.

## TROISIÈME PARTIE : ÉVALUATION EXPÉRIMENTALE ET RÉSULTATS

Cette troisième partie porte sur l'évaluation expérimentale du framework et de ses performances. Elle présente la construction du dataset, l'évaluation des modèles de détection, puis les résultats obtenus sur différents scénarios de compromission, avant de les comparer aux travaux existants.

## Chapitre 5 : Dataset et évaluation des modèles

### Introduction

Ce chapitre est consacré à l'évaluation expérimentale des composants du framework. Les différents scénarios de collecte et d'entraînement à travers les modules implémentés sont présentés afin d'analyser les performances du système et de valider les choix effectués lors de sa conception.

### I. Méthodologie expérimentale

Cette phase expérimentale repose sur la constitution d'un jeu de données représentatif, l'entraînement de plusieurs modèles de détection d'anomalies ainsi que l'analyse des caractéristiques extraites. Les sections suivantes présentent la construction du jeu de données ainsi que les métriques retenues pour l'évaluation des performances du système.

#### 1. Périmètre des menaces étudiées

Notre framework cible principalement les menaces produisant des artefacts observables dans la mémoire vive des machines virtuelles Windows.

- Rootkits noyau à module propre avec hooks, représentés notamment par les familles **Nidhogg** [20] et **PhantomKiller** [21] ;
- Techniques de dissimulation par DKOM (Direct Kernel Object Manipulation) ;
- Rootkits exploitant des pilotes noyau et générant des artefacts comportementaux détectables en mémoire ;
- Rootkits en espace utilisateur (userland), agissant sur les processus, bibliothèques dynamiques ou mécanismes d'interception accessibles depuis l'espace utilisateur. ;
- Malwares laissant des traces observables dans les processus, modules chargés ou bibliothèques dynamiques.

L'approche retenue repose sur l'observation des comportements et des artefacts mémoire plutôt que sur l'identification d'une famille particulière à l'aide de signatures.

#### 2. Construction du dataset CSV

La constitution du jeu de données est automatisée par un script dédié, `collect_dataset.py`, chargé d'orchestrer l'ensemble du processus de collecte. Chaque échantillon représente l'état mémoire de la machine virtuelle à un instant donné.

Le jeu de données final comprend **358 échantillons**, dont **262 observations saines** (Clean) et **96 observations compromises** (Infected). Les échantillons clean correspondent à des captures réalisées sur un système sain soumis à différents scénarios d'activité. Cette diversité vise à représenter la variabilité normale du fonctionnement d'un système et à éviter que le modèle n'associe à tort une activité légitime inhabituelle à un comportement malveillant. Les échantillons infected correspondent à des captures réalisées après le chargement de rootkits noyau et userland dans la machine virtuelle. Plusieurs familles de rootkits ont été employées afin d'obtenir des comportements malveillants variés et de réduire le risque d'un apprentissage trop spécifique à une implémentation particulière.

Le jeu de données est scindé en deux ensembles disjoints : D'abord le jeu de test est prélevé en premier : **20 % des observations saines et 30 % des observations compromises**, soit **52 dumps sains et 29 dumps compromis**, pour un total de **81 échantillons**. Ce jeu conserve ses étiquettes, qui sont utilisées **exclusivement pour le calcul des métriques d'évaluation**. Le corpus d'entraînement est constitué à partir des observations restantes. Les 210 dumps sains résiduels y sont intégralement versés, auxquels sont ajoutés 4 dumps compromis, tirés aléatoirement parmi les observations restantes, de manière à obtenir une contamination contrôlée de 1,87 %, conforme à l'hypothèse de contamination  $\hat{v}$  retenue. Les étiquettes ne sont ici mobilisées que pour cette étape de constitution du corpus, simulant un taux de contamination réaliste ; elles n'interviennent à aucun moment dans l'apprentissage du modèle.

Les 63 dumps compromis non retenus demeurent inutilisés. Leur inclusion porterait la contamination du corpus à 24,2 %, proportion à laquelle les observations compromises constituent une région dense de l'espace et sont intégrées par le modèle à sa notion de normalité, rendant **l'apprentissage one-class inopérant**.

| Ensemble     | Composition                                             | Étiquettes | Usage                                                            |
|--------------|---------------------------------------------------------|------------|------------------------------------------------------------------|
| Entraînement | 214 dumps : 210 sains + 4 compromis ( $\tau = 1,87\%$ ) | Retirées   | Apprentissage du domaine de normalité et calibration du seuil    |
| Test         | 81 dumps : 52 sains + 29 compromis                      | Conservées | Calcul des métriques d'évaluation                                |
| Non utilisés | 63 dumps compromis                                      | —          | Écartés pour maintenir la contamination sous le seuil de rupture |

Tableau 5.1. Répartition du dataset

## II. Évaluation des caractéristiques extraites

Avant d'évaluer les performances des modèles de détection, il est nécessaire d'analyser le comportement des différentes caractéristiques extraites à partir des dumps mémoire. Cette étape permet d'identifier les indicateurs les plus discriminants et d'évaluer leur pertinence dans le contexte des systèmes Windows modernes.

### 1. Analyse des premières caractéristiques

Les premières expérimentations portent sur des caractéristiques issues de travaux antérieurs sur la détection des rootkits par analyse mémoire. Certaines correspondent aux indicateurs exploités par TKRD, notamment les modules cachés, les hooks IRP, les hooks SSDT et les callbacks anormaux, tandis que d'autres sont complémentaires et décrivent davantage le contexte général du système, comme les nombres de modules, threads, drivers et callbacks.

| Caractéristique                | Systèmes sains (262) | Systèmes compromis (96) | Observation                             |
|--------------------------------|----------------------|-------------------------|-----------------------------------------|
| <b>hidden_modules</b>          | 0                    | 0                       | Aucun pouvoir discriminant              |
| <b>orphan_threads</b>          | 24                   | 11 – 27                 | Forte variabilité,<br>peu discriminante |
| <b>drivers_hidden</b>          | 1                    | 1 – 6                   | Faible pouvoir discriminant             |
| <b>irp_hooks</b>               | 0 – 18               | 0 – 32                  | Très dépendante du contexte             |
| <b>device_attachments_susp</b> | 0                    | 0                       | Jamais déclenchée                       |
| <b>ssdt_hooks</b>              | 0                    | 0                       | Technique neutralisée<br>par PatchGuard |
| <b>callbacks_suspects</b>      | 3 – 4                | 2 – 3                   | Faiblement discriminante                |
| <b>timers_suspects</b>         | 1                    | 0 – 2                   | Faiblement discriminante                |
| <b>total_modules</b>           | 159 – 162            | 158 – 163               | Variable contextuelle                   |
| <b>total_threads</b>           | 804 – 1067           | 563 – 1130              | Dépendante de l'activité système        |
| <b>total_drivers</b>           | 129                  | 129 – 136               | Variable contextuelle                   |
| <b>total_callbacks</b>         | 153 – 154            | 142 – 158               | Variable contextuelle                   |

Tableau 5.2. Analyse des caractéristiques historiques

L'analyse de ces premières caractéristiques que nous appellerons **historiques** met en évidence une capacité de discrimination limitée sur **Windows 10 x64**. Plusieurs indicateurs demeurent constamment nuls, tandis que d'autres présentent des distributions très proches entre les systèmes sains et les systèmes compromis, ou restent fortement influencés par le contexte d'exécution de la machine virtuelle. Ces observations s'expliquent en partie par l'évolution des mécanismes de protection du noyau, notamment **PatchGuard**, qui empêche ou limite la mise en œuvre de certaines techniques de furtivité historiquement utilisées par les rootkits, mais également par le fait que plusieurs indicateurs reflètent davantage l'état général du système que la présence effective d'une compromission.

Ces résultats montrent que les caractéristiques héritées des travaux antérieurs ne constituent plus, à elles seules, une base suffisamment robuste pour la détection des rootkits sur les versions modernes de Windows. Ils justifient ainsi l'introduction de nouvelles caractéristiques mieux adaptées aux mécanismes d'attaque contemporains, exploitées conjointement au sein des couches d'observation noyau et espace utilisateur afin d'améliorer les performances du moteur d'apprentissage automatique.

## 2. Analyse des caractéristiques rootkit noyau

Ces indicateurs reposent principalement sur l'analyse de la provenance des pilotes, de leur activité noyau et de leur conformité vis-à-vis du comportement attendu du système.

| Caractéristique                 | Systèmes sains | Systèmes compromis | Observation                                  |
|---------------------------------|----------------|--------------------|----------------------------------------------|
| <b>drivers_outside_system32</b> | 0              | 1                  | Fortement discriminante                      |
| <b>irp_hooks_suspicious</b>     | 0              | 1 – 6              | Fortement discriminante                      |
| <b>callbacks_suspicious</b>     | 0              | 1 – 3              | Fortement discriminante                      |
| <b>device_attach_suspicious</b> | 0              | 1 – 2              | Fortement discriminante                      |
| <b>drivers_dos_device_path</b>  | 0              | 1                  | Fortement discriminante                      |
| <b>hidden_processes_active</b>  | 0              | 1                  | Discriminante sur les rootkits actifs (DKOM) |

Tableau 5.3. Analyse des caractéristiques noyau

Contrairement aux caractéristiques historiques, les caractéristiques de détection des compromissions noyau présentent un pouvoir discriminant nettement supérieur. Les cinq

premières caractéristiques produisent systématiquement des valeurs nulles sur les systèmes sains, tandis qu'elles sont activées sur la grande majorité des systèmes compromis.

La caractéristique **hidden\_processes\_active**, quant à elle, n'est activée que lorsque le rootkit met effectivement en œuvre une technique de dissimulation de processus, ce qui en fait un indicateur complémentaire du comportement malveillant.

Ces résultats montrent que les indicateurs fondés sur la provenance des pilotes, les interactions anormales avec les structures du noyau ainsi que les traces laissées par les mécanismes de dissimulation sont mieux adaptées à la détection des rootkits sur les versions modernes de Windows. Ils constituent ainsi une base robuste pour l'apprentissage du comportement normal et la détection des anomalies au sein du moteur d'apprentissage automatique proposé.

### 3. Analyse des caractéristiques userland

Les caractéristiques de la couche espace utilisateur (userland) complètent les indicateurs décrivant les artefacts du noyau en mettant en évidence des comportements observables au niveau des processus et des bibliothèques dynamiques.

| Caractéristique                | Systèmes sains | Systèmes compromis | Observation                                                                 |
|--------------------------------|----------------|--------------------|-----------------------------------------------------------------------------|
| <b>procs_suspicious_path</b>   | 0              | 0 – 1              | Peu fréquente mais fortement discriminante lorsqu'elle est présente         |
| <b>procs_no_cmdline</b>        | 4              | 1 – 16             | Pouvoir discriminant modéré, dépend du contexte d'exécution                 |
| <b>dlls_suspicious_path</b>    | 0              | 0 – 1              | Peu fréquente mais fortement discriminante lorsqu'elle est présente         |
| <b>ldrmodules_inconsistent</b> | 3              | 2 – 27             | Très discriminante, traduit des incohérences dans le chargement des modules |

Tableau 5.4. Analyse des caractéristiques de la couche espace utilisateur

Contrairement aux caractéristiques de détection des compromissions noyau, les caractéristiques de la couche espace utilisateur présentent des comportements plus dépendants du contexte d'exécution de la machine virtuelle. Certaines, telles que **procs\_suspicious\_path** et **dlls\_suspicious\_path**, sont rarement activées, mais constituent des indicateurs particulièrement pertinents lorsqu'elles le sont ; un exemple représentatif d'élément déclencheur est présenté en



annexe (voir Figure A.2). À l'inverse, **procs\_no\_cmdline** est observée aussi bien sur les systèmes sains que compromis et doit être interprétée avec les autres caractéristiques. Enfin, **ldrmodules\_inconsistent** présente une augmentation marquée sur les systèmes compromis, ce qui en fait l'indicateur le plus discriminant de cette couche.

#### 4. Analyse globale du jeu de données

L'analyse individuelle des différentes caractéristiques met en évidence des comportements nettement différenciés selon leur nature. Afin de confirmer ces observations, une analyse globale du jeu de données a été réalisée en évaluant le pouvoir discriminant de chaque caractéristique au moyen de deux mesures complémentaires (voir Figure 5.1)

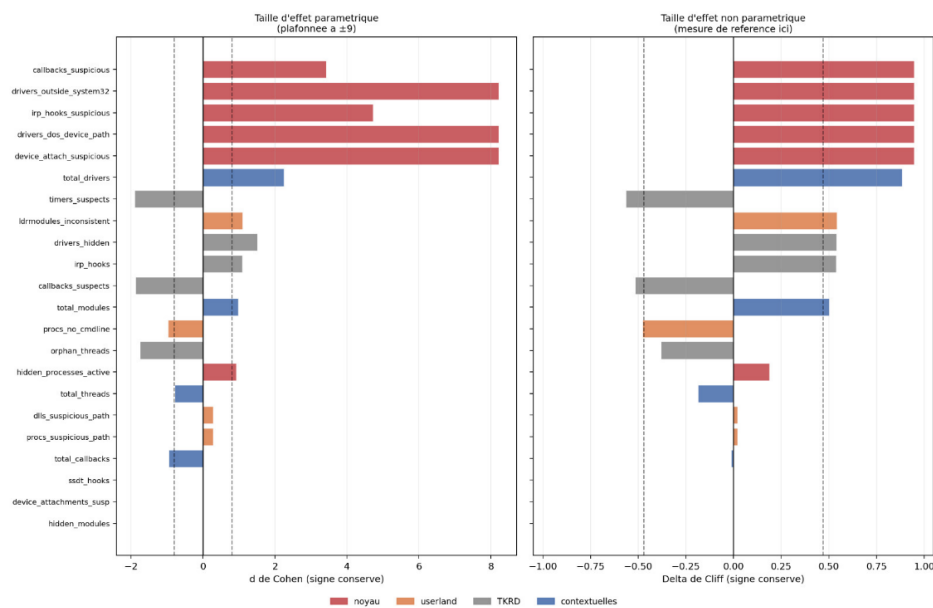


Figure 5.1. Présentation du pouvoir discriminant par caractéristique.

Le **coefficient  $d$  de Cohen**, mesure paramétrique de la taille d'effet, compare les moyennes des deux classes (saine et infectée) en rapportant leur différence à leur dispersion commune, estimée par l'écart-type poolé. Dans le corpus étudié, plusieurs caractéristiques prennent exactement la valeur zéro pour l'ensemble des observations saines : **drivers\_outside\_system32**, **drivers\_dos\_device\_path**, **device\_attach\_suspicious**, **irp\_hooks\_suspicious** et **callbacks\_suspicious**. Leur variance étant nulle dans la classe saine, la dispersion poolée n'est portée que par la classe infectée, ce qui conduit à des valeurs de coefficient très élevées, jusqu'à 8,22. Ces valeurs traduisent davantage l'absence de variabilité dans une classe qu'une taille d'effet interprétable au sens usuel, mais elles mettent en évidence des caractéristiques fortement discriminantes, dont les valeurs diffèrent nettement entre les deux classes.

Le **delta de Cliff**, mesure non paramétrique de la taille d'effet, repose quant à lui sur la comparaison de toutes les paires formées d'une observation saine et d'une observation infectée. Il indique la tendance d'une classe à présenter des valeurs supérieures à l'autre. Une valeur de (+1) traduit une séparation parfaite en faveur de la classe infectée, une valeur nulle indique l'absence de différence systématique entre les deux classes, tandis qu'une valeur négative traduit un effet en sens inverse. Ainsi, une barre orientée vers la droite correspond à une caractéristique dont les valeurs sont globalement plus élevées sur les systèmes infectés, tandis qu'une barre orientée vers la gauche indique des valeurs plus élevées sur les systèmes sains.

La lecture conjointe des deux panneaux met en évidence trois comportements distincts. Premièrement, les indicateurs noyau proposés se détachent nettement sur le panneau de droite, avec un delta de Cliff proche de sa valeur maximale. Leurs barres présentent d'ailleurs une longueur identique, ce qui indique qu'ils produisent un effet de séparation comparable entre les deux classes et se déclenchent conjointement sur les mêmes observations infectées. Deuxièmement, plusieurs caractéristiques issues des travaux antérieurs présentent un effet orienté vers la gauche, traduisant des valeurs globalement plus élevées sur les systèmes sains que sur les systèmes infectés. Enfin, trois caractéristiques n'apparaissent sur aucun des deux panneaux : leur effet est nul, car elles prennent une valeur identique sur l'ensemble du corpus et n'apportent, par conséquent, aucune information discriminante exploitable pour distinguer les deux classes.

Cette analyse met ainsi en évidence des différences nettes de comportement entre les caractéristiques et permet de mieux apprécier leur capacité respective à distinguer les systèmes sains des systèmes infectés.

### III. Évaluation des modèles de détection

Après avoir étudié le comportement des différentes caractéristiques extraites, il convient d'évaluer les performances des modèles retenus pour le moteur de détection.

L'objectif est de déterminer leur capacité à distinguer les systèmes sains des systèmes compromis à partir des vecteurs de caractéristiques produits par le module d'extraction.

L'évaluation des modèles repose sur plusieurs métriques :

| Métrique | Description                          | Formule                         |
|----------|--------------------------------------|---------------------------------|
| Accuracy | Proportion de prédictions correctes. | $(VP + VN)/(VP + VN + FP + FN)$ |

|                     |                                                                             |                                                                                           |
|---------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Precision</b>    | Proportion de prédictions positives correctes.                              | $VP/(VP + FP)$                                                                            |
| <b>Recall (TPR)</b> | Proportion de compromissions correctement détectées.                        | $VP/(VP + FN)$                                                                            |
| <b>FPR</b>          | Proportion d'observations saines incorrectement classées comme compromises. | $FP/(FP+VN)$                                                                              |
| <b>F1-score</b>     | Compromis entre précision et rappel.                                        | $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$ |
| <b>ROC-AUC</b>      | Capacité à distinguer les observations saines et compromises.               | $\int TPR d(FPR)$                                                                         |

Tableau 5.5. Métriques d'évaluation

Avec : VP = vrais positifs, VN = vrais négatifs, FP = faux positifs et FN = faux négatifs.

Ces indicateurs permettent de comparer objectivement les différents modèles étudiés et d'identifier la solution la plus adaptée au contexte de détection considéré.

Conformément aux choix présentés au chapitre 3, les performances des trois algorithmes choisis : Isolation Forest (IF), Local Outlier Factor (LOF) et One-Class Support Vector Machine (OCSVM) sont évaluées à l'aide des métriques présentées dans le Tableau 5.5. L'ensemble des caractéristiques a été soumis à des expérimentations conduites par approche **gloutonne** afin de restreindre l'évaluation des performances des modèles à des sous-ensembles méritant d'être observées. Le vecteur de dix caractéristiques (noyau et userland) constitue le point de départ. À chaque étape, chacune des douze caractéristiques héritées encore disponibles est ajoutée individuellement, chaque algorithme est réentraîné sur le corpus non trié et évalué sur le jeu de test, et celle qui maximise le rappel est retenue ; les égalités étant départagées par l'écart-type croissant, puis par le taux de faux positifs.

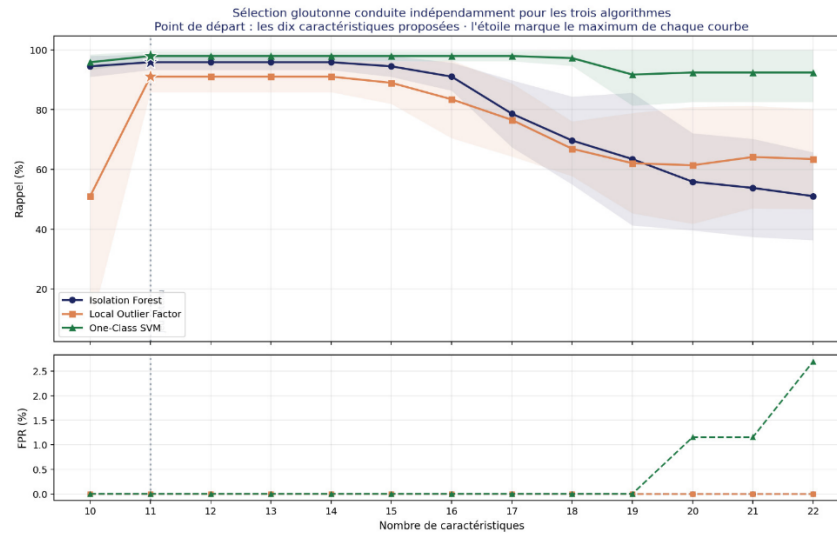


Figure 5.2. Résultats de l'expérience de sélection gloutonne.

Les trois algorithmes atteignent leur rappel maximal au rang **onze** : **97,93 %** pour One-Class SVM, **95,86 %** pour Isolation Forest et **91,03 %** pour Local Outlier Factor. Ils y sélectionnent pourtant des caractéristiques distinctes respectivement `drivers_hidden`, `total_drivers` et `total_threads` et convergent donc vers un même palier par des chemins différents. Au-delà, les performances se dégradent régulièrement pour les trois algorithmes. Trois sous-ensembles sont donc retenus pour la suite de l'analyse : le vecteur réduit de dix caractéristiques proposées, le vecteur de onze caractéristiques issues de cette sélection, et l'ensemble complet des vingt-deux caractéristiques extraites lors de l'analyse forensique. Les sections suivantes présentent successivement les résultats obtenus pour chaque algorithme, avant d'établir une comparaison globale permettant d'identifier la solution la plus adaptée au contexte étudié.

## 1. Isolation Forest

### a. Paramétrage et entraînement

Plusieurs hyperparamètres ont été ajustés au cours des expérimentations. Ils concernent notamment :

| Paramètre           | Valeur utilisée | Description                                                                                                           |
|---------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>n_estimators</b> | 800             | Nombre d'arbres constituant la forêt d'isolation.                                                                     |
| <b>max_samples</b>  | 0.8             | Nombre d'observations échantillonnées pour chaque arbre.                                                              |
| <b>random_state</b> | 42              | Valeur utilisée pour initialiser le générateur aléatoire et garantir la reproductibilité des résultats expérimentaux. |

Tableau 5.6. Tableau récapitulatif de paramétrage IF

Les différentes configurations ont été évaluées au moyen d'une validation croisée à cinq plis, afin de sélectionner celles offrant le meilleur compromis entre capacité de détection, robustesse et performances globales.

#### b. Résultats obtenus

Les performances obtenues par le modèle sont présentées sur la Figure 5.3 ci-dessous :

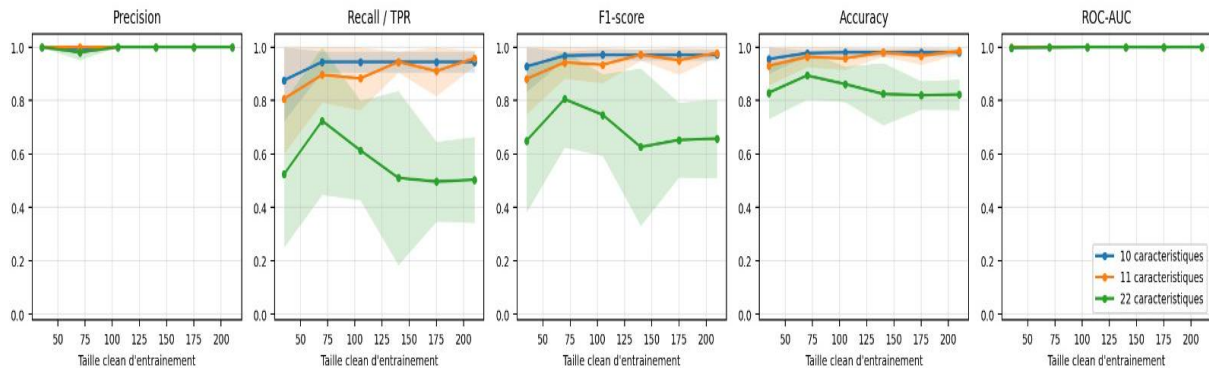


Figure 5.3. Résultats de métriques IF

Les résultats montrent que les vecteurs réduits offrent de meilleures performances que le vecteur étendu, quelle que soit la taille du jeu d'entraînement. L'écart est particulièrement marqué sur le rappel, qui atteint **95,86 %** avec onze caractéristiques et **94,48 %** avec dix, contre **50,34 %** avec vingt-deux, ainsi que sur le score F1 et l'accuracy. La précision demeure parfaite et le ROC-AUC égal à 1 pour les trois configurations. Ces différences de performance s'expliquent par la nature des caractéristiques additionnelles. Les indicateurs TKRD et les compteurs contextuels décrivent l'état d'activité du système plutôt que la présence d'une compromission ; leur inclusion rend les scores plus dispersés, ce qui remonte le seuil sans pour autant améliorer la séparation entre observations saines et compromises.

## 2. Local Outlier Factor

### a. Paramétrage et entraînement

Les hyperparamètres présentés ci-dessous contrôlent la définition du voisinage local et le calcul de la densité utilisée pour identifier les anomalies.

| Paramètre          | Valeur utilisée | Description                                                |
|--------------------|-----------------|------------------------------------------------------------|
| <b>n_neighbors</b> | 40              | Nombre de voisins utilisés pour estimer la densité locale. |

|                |           |                                                                       |
|----------------|-----------|-----------------------------------------------------------------------|
| <b>metric</b>  | manhattan | Distance utilisée entre les observations.                             |
| <b>novelty</b> | True      | Autorise la détection de nouvelles observations après l'entraînement. |

Tableau 5.7. Tableau récapitulatif de paramétrage LOF

Ces hyperparamètres ont été déterminés à partir de plusieurs expérimentations successives puis validés par validation croisée à cinq plis. Le choix final privilégie un équilibre entre qualité de détection, robustesse du voisinage local et stabilité des performances observées.

#### b. Résultats obtenus et analyse

Les performances obtenues par le modèle LOF sont présentées sur la Figure 5.4 suivante :

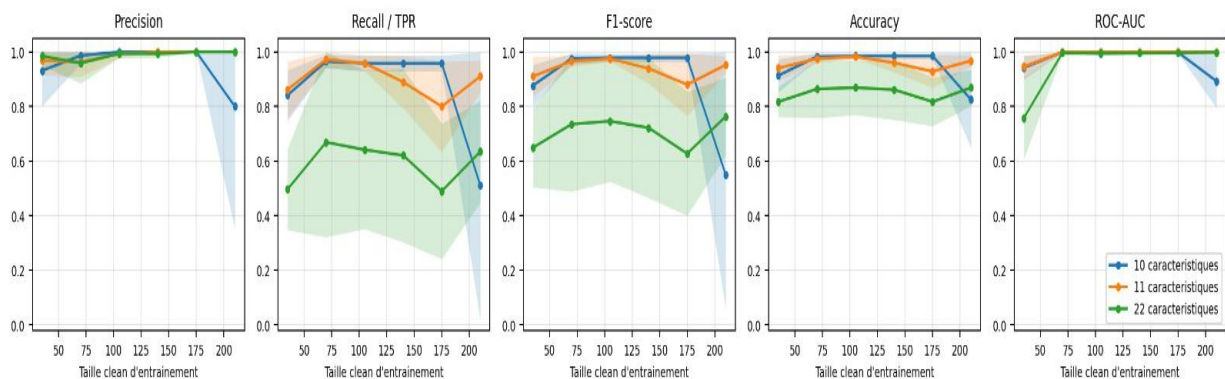


Figure 5.4. Résultats de métriques LOF

Les résultats montrent que Local Outlier Factor bénéficie davantage du vecteur de onze caractéristiques que des deux autres. Celui-ci atteint un rappel de **91,03 %**, contre **51,03 %** pour le vecteur réduit et **63,45 %** pour le vecteur étendu. La précision reste élevée et le taux de faux positifs nul dans les trois configurations. Toutefois, les scores de F1 et d'accuracy demeurent inférieurs à ceux obtenus avec l'algorithme Isolation Forest. Le pouvoir de classement suit la même progression, avec un ROC-AUC de 1 pour le vecteur de onze caractéristiques, contre 0,893 pour le vecteur réduit et 0,998 pour le vecteur étendu.

### 3. One-Class Support Vector Machine

#### a. Paramétrage et entraînement

Les éléments de paramétrage présentés ci-dessous influencent la forme de la frontière représentant le comportement normal du système et la sensibilité du détecteur aux observations atypiques :

| Paramètre     | Valeur utilisée | Rôle                                                                                                                                       |
|---------------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>kernel</b> | rbf             | Fonction noyau utilisée pour modéliser la frontière du comportement normal.                                                                |
| <b>gamma</b>  | 0.05            | Contrôle l'influence de chaque observation sur la frontière de décision.                                                                   |
| <b>nu</b>     | 0.05            | Contrôle la sensibilité du modèle en fixant la proportion maximale d'observations d'entraînement pouvant être considérées comme atypiques. |

Tableau 5.8. Tableau récapitulatif de paramétrage OCSVM

Ces hyperparamètres ont été déterminés à partir de plusieurs expérimentations successives, puis évalués sur plusieurs découpages du jeu de données. La configuration finale a été retenue au regard des performances obtenues sur les jeux de calibration et de test, en recherchant un compromis entre capacité de détection, stabilité et limitation des faux positifs.

#### b. Résultats obtenus

L'évaluation du modèle One-Class SVM a été réalisée sur le même protocole expérimental que celui appliqué aux autres modèles. Les performances obtenues sont présentées dans la figure Figure 5.5 suivante :

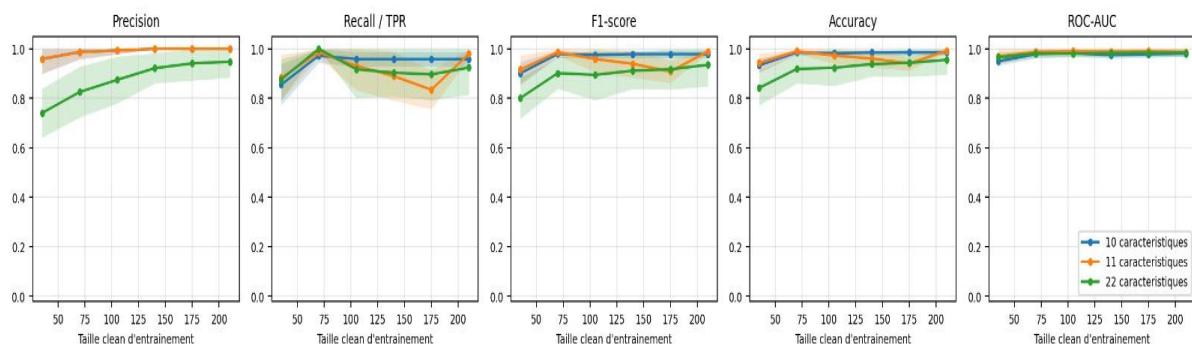


Figure 5.5. Résultats de métriques OCSVM

Les résultats montrent que **les vecteurs réduits offrent de meilleures performances que le vecteur étendu**. Le vecteur à onze caractéristiques atteint un rappel de **97,93 %**, contre **95,86**

% pour celui à dix caractéristiques et **92,41 %** pour le vecteur à vingt-deux caractéristiques. Les deux vecteurs réduits présentent également un **taux de faux positifs nul** et une précision parfaite, tandis que le vecteur étendu affiche un taux de faux positifs de **2,69 %** et une précision de **94,77 %**. L'ajout de la onzième caractéristique améliore ainsi les performances du modèle : le rappel passe de **95,86 % à 97,93 %**, l'accuracy de **98,52 % à 99,26 %** et le ROC-AUC de **0,981 à 0,987**, tandis que la précision reste de **100 %**. À l'inverse, l'ajout de caractéristiques supplémentaires dans le vecteur étendu entraîne une dégradation des performances.

#### 4. Analyse comparative et composition finale du détecteur

Le tableau ci-dessous récapitule les performances des trois modèles sur les trois jeux de caractéristiques, sous l'hypothèse de contamination retenue. Les métriques rapportées correspondent à la moyenne de cinq évaluations (5 splits).

| Caractéristiques           | Modèle       | Accuracy (%) | F1-score     | FPR (%)     | ROC-AUC      | Recall / TPR (%) |
|----------------------------|--------------|--------------|--------------|-------------|--------------|------------------|
| <b>10 caractéristiques</b> | IF           | 98,02        | 0,971        | 0,00        | 1,0000       | 94,48            |
|                            | LOF          | 82,47        | 0,550        | 0,00        | 0,893        | 51,03            |
|                            | OCSVM        | 98,52        | 0,979        | 0,00        | 0,981        | 95,86            |
| <b>11 caractéristiques</b> | IF           | 98,52        | 0,979        | 0,00        | 1,0000       | 95,86            |
|                            | LOF          | 96,79        | 0,952        | 0,00        | 1,0000       | 91,03            |
|                            | <b>OCSVM</b> | <b>99,26</b> | <b>0,990</b> | <b>0,00</b> | <b>0,987</b> | <b>97,93</b>     |
| <b>22 caractéristiques</b> | IF           | 82,22        | 0,657        | 0,00        | 1,0000       | 50,34            |
|                            | LOF          | 86,91        | 0,763        | 0,00        | 0,998        | 63,45            |
|                            | OCSVM        | 95,56        | 0,935        | 2,69        | 0,984        | 92,41            |

Tableau 5.9. Performances des trois modèles sur les trois jeux de caractéristiques.

Ces résultats montrent que One-Class SVM associé au vecteur de onze caractéristiques obtient les meilleures performances globales : un rappel de **97,93 %** avec un taux de faux positifs nul et une précision de 100 %. C'est la seule configuration à réunir simultanément le rappel le plus élevé, l'absence de fausses alertes et l'écart-type le plus faible entre découpages.

One-Class SVM sur onze caractéristiques a donc été retenu comme moteur de détection du framework, la limitation des fausses alertes conditionnant l'exploitabilité opérationnelle du détecteur.



| Paramètre                                | Valeur                          |
|------------------------------------------|---------------------------------|
| Corpus d'entraînement                    | 214 observations non étiquetées |
| Observations compromises injectées       | 4                               |
| Contamination effective ( $\tau$ )       | 1,87 %                          |
| Hypothèse de contamination ( $\hat{v}$ ) | 2 %                             |
| Quantile retenu                          | 98 %                            |
| Seuil exporté                            | <b>-3,553532</b>                |

Tableau 5.10. Paramètres de calibration du seuil de décision du modèle One-Class SVM

### Conclusion

Ce chapitre a présenté l'évaluation expérimentale des éléments du framework. Les résultats obtenus confirment la pertinence des caractéristiques retenues et justifient le choix de One-Class SVM comme moteur de détection. Le chapitre suivant présente les résultats du détecteur final et les compare aux travaux existants.

## Chapitre 6 : Résultats et analyse

### Introduction

Ce chapitre présente les résultats obtenus avec la version finale du framework. L'objectif est d'évaluer son comportement face à différents scénarios de compromission, puis de comparer l'approche proposée à des travaux existants avant d'en discuter les forces et les limites.

### I. Expérimentation du détecteur final

Afin de faciliter l'analyse des résultats produits par le détecteur, une interface de supervision a été développée à l'aide du framework python Streamlit. Elle centralise les informations générées lors de chaque analyse et permet de visualiser les caractéristiques extraites, le verdict du moteur d'apprentissage automatique, la probabilité estimée de compromission ainsi que la contribution des couches noyau et espace utilisateur à la décision finale. Elle constitue également un support pratique pour l'interprétation et la validation expérimentale des résultats présentés dans cette section.

L'évaluation du détecteur final a été réalisée à travers plusieurs scénarios représentatifs de compromission. Ces expérimentations visent à vérifier la capacité du système à détecter aussi bien la présence de rootkits noyau que certaines activités malveillantes en espace utilisateur.

#### 1. Détection de rootkits noyau

##### a. Détection de la présence d'un rootkit chargé

La première série d'expérimentations vise à vérifier la capacité du framework à détecter un rootkit simplement chargé en mémoire, sans qu'une action malveillante particulière ne soit nécessairement exécutée. Cette situation est particulièrement importante car de nombreux rootkits modernes demeurent passifs pendant une grande partie de leur cycle de vie afin de limiter leur exposition.

Afin d'évaluer la capacité de généralisation de notre modèle, une expérimentation a été réalisée à l'aide d'un troisième pilote malveillant qui n'avait jamais été utilisé durant la phase d'évaluation de capacité de détection du modèle après l'entraînement : ThrottleStop.sys. Ce pilote exploite une vulnérabilité permettant d'obtenir des primitives d'accès mémoire noyau et peut être utilisé comme support pour diverses manipulations privilégiées du système.

Dans un premier temps, avant le chargement du pilote malveillant, les six caractéristiques de la couche noyau demeurent nulles, traduisant l'absence d'anomalies observables au niveau du noyau. Seules quelques caractéristiques de la couche espace utilisateur présentent de faibles valeurs, correspondant au fonctionnement normal du système d'exploitation. Le moteur de détection One-Class SVM classe alors la machine comme **SAIN** avec une confiance de **98,4 %**, soit un risque d'infection estimé à **1,6 %** (voir Figure 6.1). La couche espace utilisateur est identifiée comme la plus influente, les variations observées provenant exclusivement des caractéristiques userland.

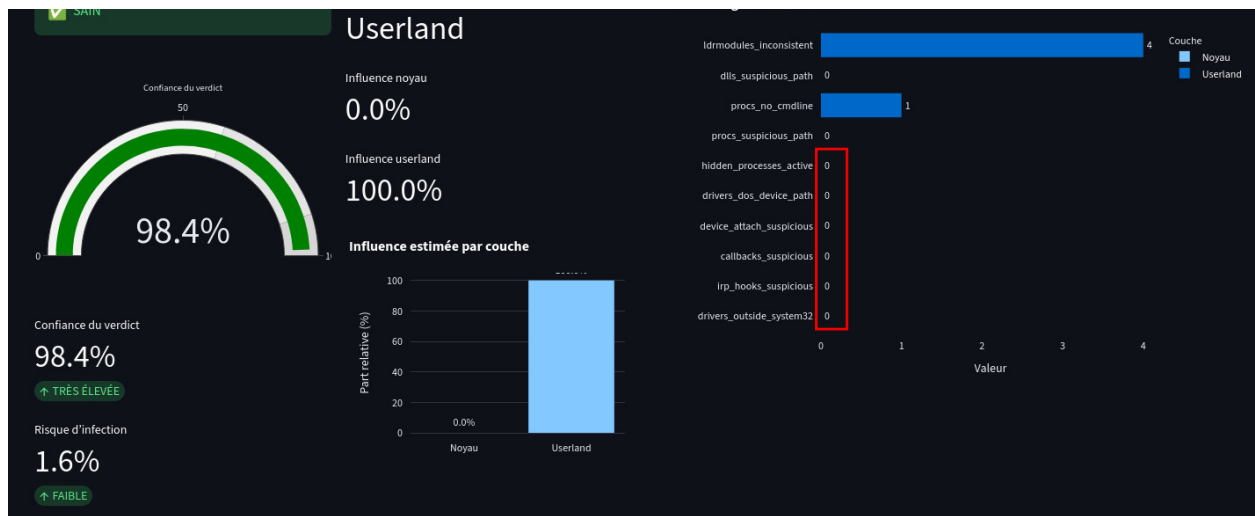


Figure 6.1. Résultats de détection état SAIN

Une fois le pilote chargé dans la machine virtuelle Windows, Comme illustré avec la Figure 6.2 ci-dessous :

```
PS C:\Users\framework\Desktop\CVE-2025-7771\target\release> sc.exe start ThrottleStop

SERVICE_NAME: ThrottleStop
        TYPE               : 1  KERNEL_DRIVER
        STATE                : 4  RUNNING
                           (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE       : 0  (0x0)
        SERVICE_EXIT_CODE   : 0  (0x0)
        CHECKPOINT           : 0x0
        WAIT_HINT            : 0x0
        PID                 : 0
        FLAGS                 :
PS C:\Users\framework\Desktop\CVE-2025-7771\target\release>
```

Figure 6.2. Activation du pilote ThrottleStop.sys

On constate que plusieurs caractéristiques de la couche noyau s'activent, notamment celles relatives aux pilotes chargés hors des chemins système, aux routines IRP suspectes, aux callbacks et aux attachements de périphériques. Des indicateurs de la couche espace utilisateur

sont également activés. Le moteur One-Class SVM identifie alors la machine comme compromise et produit un verdict **INFECTED** avec un risque d'infection estimé à **99 %**, confirmant la présence d'un comportement fortement anormal (voir Figure 6.3).



Figure 6.3. Résultats de détection état INFECTED

Cette expérimentation est particulièrement intéressante car le pilote utilisé n'appartenait pas au jeu de données. Les résultats montrent ainsi que le détecteur ne mémorise pas simplement les caractéristiques propres à Nidhogg ou PhantomKiller, mais qu'il est capable de détecter une compromission à partir d'indices comportementaux communs à différentes familles de rootkits. Cette capacité de généralisation constitue l'un des objectifs principaux de l'approche proposée.

#### b. Détection comportementale

Une seconde expérimentation a été réalisée afin de valider la détection d'un comportement malveillant actif (voir Figure 6.4). Après le chargement du pilote, un processus **Notepad.exe** a été masqué à l'aide d'une technique de type **DKOM (Direct Kernel Object Manipulation)** [22], consistant à le retirer des mécanismes classiques d'énumération des processus.

```

PS C:\Users\framework\Desktop\CVE-2025-7771\target\release> notepad.exe
PS C:\Users\framework\Desktop\CVE-2025-7771\target\release> .\CVE-2025-7771.exe hide notepad.exe
[+] Kernel Base VA: 0xFFFFF8041B81B000
[*] locating PsInitialSystemProcess :
    Module ntoskrnl.exe loaded
    PsInitialSystemProcess at : 0x7FF6A5A8C420
    Offset: 13616160
[+] Building memory map...
    -> Kernel Base (Physical Address) : 0x03400000
    -> PsInitialSystemProcess (Physical Address) : 0x040FC420
[+] Driver initialized and ready for operation, Handle : 0xc864ffc18
[+] System EPROCESS VA: 0xFFFFC98D94281040
[*] Hiding PID: 1360
    -> EPROCESS for PID 1360 at VA: 0xffffc98d9b6bc080
[+] Writing 8 bytes at physical address -> 0x085654C8 : 88 07 18 96 8D C9 FF FF
[+] Writing 8 bytes at physical address -> 0x02E12790 : C8 24 9B 94 8D C9 FF FF
[+] Writing 8 bytes at physical address -> 0x4D6FD4C8 : C8 C4 6B 9B 8D C9 FF FF
[+] Writing 8 bytes at physical address -> 0x4D6FD4D0 : C8 C4 6B 9B 8D C9 FF FF
[+] Process hidden!
[*] Cleaning up ...
[*] Driver Handle closed!
PS C:\Users\framework\Desktop\CVE-2025-7771\target\release>

```

Figure 6.4. Illustration attaque DKOM

Après cette opération, une nouvelle capture mémoire est réalisée. L'analyse met en évidence l'activation de la caractéristique **hidden\_processes\_active** (voir Figure 6.5), indiquant la présence d'un processus détecté dans les structures mémoire mais absent des mécanismes classiques d'énumération. Cette observation confirme la détection du comportement de dissimulation mis en œuvre par le rootkit.

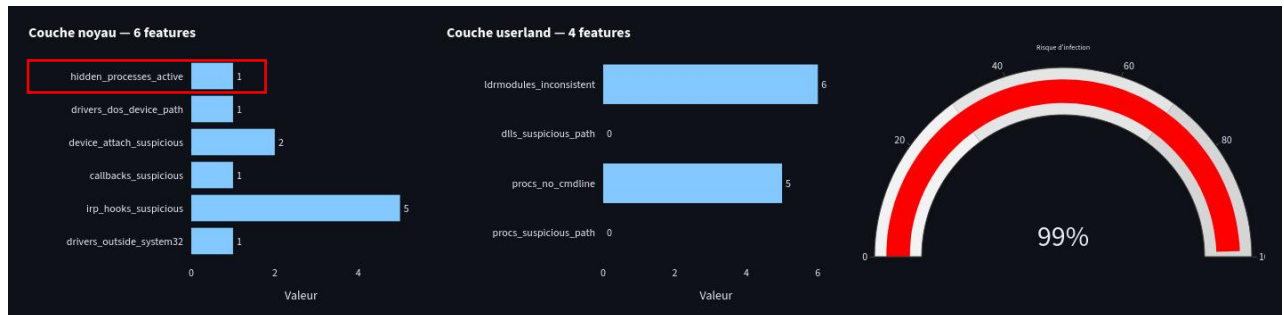


Figure 6.5. Résultats détection DKOM

## 2. Détection de menaces en espace utilisateur

Le framework ne se limite pas à l'analyse des artefacts noyau. Il exploite également des caractéristiques issues de l'espace utilisateur (voir Figure 6.6) afin de détecter des comportements susceptibles de révéler une compromission ne nécessitant pas l'utilisation d'un pilote malveillant.

```

[OK] Stealth Mode active (process + files hidden)
PS C:\Users\Framework\AppData\Local\Temp\lab> .\rootkit_debug.exe
=====
XvX ROOTKIT v3.0 - STARTING
=====
[INFO] [1/8] Running evasion checks...
[OK] Evasion checks passed
[WARN] [SKIP] NTDLL unhooking disabled for lab VM
[Persistence] Scheduled task created: \Microsoft\Windows\NetTrace\GatherNetworkInfo
[Persistence] COM hijack installed for CLSID: {00021401-0000-0000-C000-000000000046}
[OK] Persistence installed
[-] c2_config.txt not found: C:\Users\Framework\AppData\Local\Temp\lab\c2_config.txt
[*] Using default C2 URL
[-] Connection failed: http://127.0.0.1:8080
[!] C2 unavailable - running in standalone mode
[i] Rootkit features available without C2:
  - NTDLL unhooking (active - anti-EDR)
  - ETW/AMSI bypass (active - anti-telemetry)
  - Process hiding (processHooks.dll)
  - File hiding (fileHooks.dll)
  - Registry hiding (registryHooks.dll)

[+] Rootkit active!
[i] Daemon mode - Running in background
[*] Activating stealth mode...
[INFO] Found taskmgr.exe PID: 5936
[SUCCESS] DLL injected: C:\Users\Framework\AppData\Local\Temp\lab\processHooks.dll -> PID 5936
[+] Injected processHooks.dll into taskmgr.exe (PID 5936)
[INFO] Found explorer.exe PID: 4536
[SUCCESS] DLL injected: C:\Users\Framework\AppData\Local\Temp\lab\fileHooks.dll -> PID 4536
[+] Injected fileHooks.dll into explorer.exe (PID 4536)
[OK] Stealth mode active (process + files hidden)
PS C:\Users\Framework\AppData\Local\Temp\lab>

```

Figure 6.6. Chargement rootkit userland

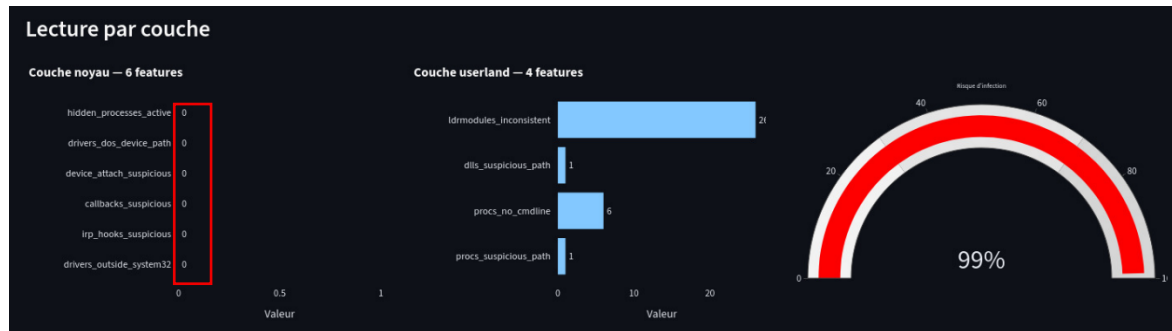


Figure 6.7. Détection d'une menace userland en activité

Les résultats sur la Figure 6.7. montrent que les caractéristiques de la couche noyau demeurent nulles, ce qui est cohérent avec la nature de la menace étudiée, celle-ci n'impliquant pas le chargement d'un pilote en mode noyau. En revanche, plusieurs caractéristiques de la couche espace utilisateur sont activées, notamment celles liées à l'exécution de processus et au chargement de bibliothèques depuis des chemins suspects. L'analyse des détails extraits confirme notamment l'exécution du processus malveillant depuis un répertoire temporaire, comportement caractéristique de ce type de rootkit (voir Figure 6.8). Ces observations permettent au détecteur d'identifier la compromission et de produire un verdict **INFECTED** avec un risque d'infection de **99 %**.

| Détails des artefacts userland |                                      |        |                     |                 |
|--------------------------------|--------------------------------------|--------|---------------------|-----------------|
| feature                        | description                          | valeur | détails_disponibles | éléments_listés |
| procs_suspicious_path          | Processus depuis chemins suspects    | 1      | oui                 | 1               |
| procs_no_cmdline               | Processus sans ligne de commande     | 5      | oui                 | 5               |
| dlls_suspicious_path           | DLL chargées depuis chemins suspects | 1      | oui                 | 1               |
| ldmodules_inconsistent         | Incohérences du chargeur Windows     | 29     | oui                 | 29              |

Les listes ci-dessous apparaissent si l'extracteur a enregistré les détails dans features\_all. Si une feature est non nulle mais sans détails, le compteur est disponible mais pas encore la liste des éléments concernés.

Processus depuis chemins suspects – valeur = 1

| # | pid  | processus      | chemin                                                                       | cmdline                                                                      | raison                  |
|---|------|----------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------|-------------------------|
| 1 | 4368 | r00tkit_debug. | "C:\Users\framework\AppData\Local\Temp\lab\deploy_package\r00tkit_debug.exe" | "C:\Users\framework\AppData\Local\Temp\lab\deploy_package\r00tkit_debug.exe" | chemin suspect: \temp\, |

Figure 6.8. Détails des artefacts userland détectés

Cette dernière expérimentation montre que le framework ne se limite pas aux seuls rootkits noyau et présente un potentiel de généralisation à d'autres malwares produisant des artefacts mémoire similaires.

## II. Analyse comparative des résultats

### 1. Comparaison avec l'approche TKRD

TKRD constitue le principal travail de référence ayant guidé la conception de notre framework. Le rapprochement mené ici poursuit deux objectifs : mesurer ce que devient une approche conçue pour Windows 7 x86 lorsqu'elle est transposée à un système moderne.

| Critère                     | TKRD                                               | Framework proposé                                         |
|-----------------------------|----------------------------------------------------|-----------------------------------------------------------|
| Environnement évalué        | Windows 7 Ultimate x86                             | Windows 10 x64 Enterprise LTSC                            |
| Plateforme cloud            | OpenStack multi-nœuds                              | OpenStack mono-nœud (DevStack)                            |
| Outil d'analyse             | Volatility 2.6                                     | Volatility 3                                              |
| Périmètre des menaces       | Rootkits noyau (NTRootkit, FURootkit et variantes) | Rootkits noyau et userland, DKOM, BYOVD, injection de DLL |
| Niveau d'observation        | Noyau                                              | Noyau et espace utilisateur                               |
| Nombre de caractéristiques  | 8                                                  | 11                                                        |
| Représentation              | Booléenne, pondérée par information mutuelle       | Compteurs entiers                                         |
| Régime d'apprentissage      | Supervisé                                          | Non supervisé (one-class)                                 |
| Étiquettes à l'entraînement | Requises                                           | Non requises                                              |
| Modèle retenu               | Random Forest                                      | One-Class SVM                                             |
| Corpus                      | 12 300 dumps (10 000 compromis / 2 300 sains)      | 358 dumps (96 compromis / 262 sains)                      |

Tableau 6.1. Positionnement par rapport à l'état de l'art



## 2. Positionnement du framework

La comparaison avec TKRD met en évidence une différence fondamentale dans la stratégie d'apprentissage. Le framework de Wang et al. repose sur un **apprentissage supervisé** : les auteurs entraînent plusieurs classifieurs à partir d'observations bénignes et malveillantes, puis retiennent Random Forest, qui obtient une accuracy de **98,6 %**, un TPR de **98,4 %**, un FPR de **7,6 %** et un ROC-AUC de **0,998** lors de la détection de nouveaux rootkits.

Le framework proposé adopte une approche différente en privilégiant un **apprentissage one-class**, apprend ainsi les caractéristiques du fonctionnement normal du système et considère comme anormales les observations qui s'en écartent. Cette stratégie permet de limiter la dépendance à un corpus d'échantillons infectés lors de l'entraînement et oriente le système vers une **détection d'anomalies** plutôt que vers une classification supervisée de familles de menaces.

Cette différence est renforcée par l'évolution de l'espace de caractéristiques. Alors que TKRD s'appuie principalement sur des indicateurs issus de l'analyse des artefacts noyau avec **Volatility 2.6**, le framework proposé exploite **Volatility 3** et combine des indicateurs noyau avec des caractéristiques issues du **userland**. Le framework ne cherche donc pas à reproduire directement TKRD, mais à **étendre son principe de détection fondée sur l'analyse mémoire vers une approche non supervisée et multi-niveaux**, adaptée à **Windows 10 x64**.

Ainsi, le principal apport du framework proposé ne réside pas uniquement dans l'amélioration des métriques de détection par rapport à TKRD. Il réside surtout dans le **changement de paradigme d'apprentissage**, le passage à une détection one-class et l'élargissement des observations du noyau vers le userland. Les résultats expérimentaux montrent que cette orientation permet d'obtenir, avec le vecteur à onze caractéristiques, un rappel de **97,93 %**, une accuracy de **99,26 %**, un **FPR nul** et un ROC-AUC de **0,987**. Ces résultats positionnent ainsi le framework comme une approche complémentaire à TKRD, davantage orientée vers la détection d'états anormaux à partir du comportement sain du système.

## III. Discussion critique

### 1. Points forts du framework

L'un des principaux atouts de notre solution réside dans son approche fondée sur l'introspection mémoire. En observant directement les artefacts présents dans la mémoire des



machines virtuelles, le framework reste indépendant des mécanismes de supervision internes au système invité et limite ainsi les possibilités de dissimulation offertes aux menaces.

Le système bénéficie également d'une architecture modulaire composée d'un module d'acquisition mémoire, d'un module d'analyse forensique, d'un moteur d'extraction de caractéristiques et d'un moteur de détection par apprentissage automatique. Cette organisation facilite l'évolution du framework et permet d'intégrer de nouvelles caractéristiques ou de nouvelles méthodes d'analyse sans remettre en cause l'ensemble de la chaîne de traitement.

Enfin, l'exploitation conjointe des caractéristiques issues des couches **noyau** et **userland** permet d'associer une détection automatisée à une meilleure interprétation des résultats. Cette organisation offre un bon compromis entre capacité de généralisation face aux nouvelles menaces et explicabilité des verdicts produits par le framework.

## 2. Positionnement scientifique du travail

Ce travail ne se limite pas à la mise en œuvre d'un outil : il apporte une contribution méthodologique sur trois plans.

**Sur le plan de la démarche scientifique**, la conception des caractéristiques a précédé le choix du modèle. Le jeu initial de vingt-deux caractéristiques regroupait les indicateurs hérités des travaux antérieurs, les compteurs contextuels et les dix caractéristiques proposées. L'analyse par le (d) de Cohen et le delta de Cliff a permis d'identifier trois groupes. Cinq des six indicateurs noyau sont nuls sur les 262 observations saines et se déclenchent sur **94,8 %** des observations compromises, avec un delta de Cliff de **0,948** et une AUC de **0,974**. Les quatre caractéristiques userland présentent un pouvoir discriminant intermédiaire mais complémentaire. Les douze restantes sont peu exploitables, en raison de leur absence de variation, d'un effet inverse ou d'un fort recouvrement entre les deux classes. La sélection finale repose ainsi sur une démarche en deux étapes : réduction des caractéristiques fortement recouvrantes, puis sélection gloutonne par réintroduction individuelle des variables écartées et réévaluation du détecteur. Cette procédure a permis de retenir une caractéristique au pouvoir discriminant individuel limité mais apportant une information complémentaire. La validation en conditions réelles a confirmé ce choix : les caractéristiques écartées concentraient les fausses alertes, tandis que les indicateurs retenus n'en ont produit aucune sur les cycles de supervision évalués.

**Sur le plan du paradigme d'apprentissage**, le choix du régime non supervisé n'est pas une contrainte subie mais une décision motivée : contrairement à une approche supervisée qui

nécessiterait un corpus labellisé de grande taille, l'approche one-class permet de caractériser un comportement normal et de détecter toute déviation, y compris face à des menaces inédites. La détection d'un pilote malveillant absent du jeu de données (ThrottleStop.sys, Chapitre 6) illustre cette capacité, sans toutefois constituer une démonstration statistique, l'observation portant sur un cas unique.

**Sur le plan de la rigueur méthodologique**, trois modèles one-class (Isolation Forest, LOF, One-Class SVM) ont été systématiquement comparés selon des critères explicites : rappel, taux de faux positifs et stabilité du seuil. Le modèle final n'ayant pas été retenu par défaut. Cette comparaison est complétée par une confrontation qualitative aux résultats de TKRD (Wang et al., 2019), l'un des principaux travaux de référence du domaine (Tableau 6.1).

### 3. Limites de l'approche

Malgré les résultats obtenus, plusieurs limites doivent être prises en considération lors de l'interprétation des performances du framework.

Tout d'abord, l'approche proposée repose sur l'introspection mémoire des machines virtuelles et nécessite une infrastructure de virtualisation compatible. Elle n'est donc pas directement applicable à des systèmes physiques et suppose que l'hôte ainsi que les composants de virtualisation demeurent intègres.

Par ailleurs, l'acquisition et l'analyse des dumps mémoire restent coûteuses en temps de calcul et en espace de stockage (environ **2,2 Go** par dump). Le délai cumulé entre la capture mémoire, l'analyse forensique, l'extraction des caractéristiques et la production du verdict, pouvant dépasser **3 minutes**, limite la capacité du framework à détecter des activités malveillantes de très courte durée survenant entre deux acquisitions.

Une autre limite concerne **la diversité du domaine de normalité appris par le modèle**. Bien que l'ensemble du jeu de données sain comprenne 262 dumps, ceux-ci correspondent à seulement **54 vecteurs de caractéristiques distincts**. Cette faible diversité des profils normaux pourrait limiter la capacité du modèle à couvrir des situations plus variées en conditions de production. Les vérifications présentées en annexe suggèrent toutefois que cette contrainte n'a pas significativement affecté les résultats rapportés. Une diversification du corpus d'entraînement sera néanmoins nécessaire lors de futures campagnes de collecte.

Les expérimentations ont également été réalisées sur un nombre limité de familles de rootkits et de scénarios légitimes, principalement sous **Windows 10 LTSC**. Bien que les résultats

obtenus soient encourageants, ils ne permettent pas de garantir une généralisation à l'ensemble des menaces ni à d'autres versions de Windows ou systèmes d'exploitation sans validations complémentaires.

Enfin, les caractéristiques exploitées par le détecteur sont construites à partir des artefacts reconstruits par **Volatility 3**. Par conséquent, une attaque ne laissant pas de traces exploitables par les plugins utilisés ou exploitant des techniques non couvertes par l'outil pourrait ne pas être détectée.

De plus, la sélection des caractéristiques repose actuellement sur une démarche manuelle et ne permet pas encore la découverte automatique de nouveaux indicateurs de compromission.

## 2. Perspectives

Les résultats obtenus dans ce travail ouvrent plusieurs perspectives d'évolution pour le framework.

Une première perspective consiste à enrichir les expérimentations par l'intégration d'un plus grand nombre de familles de rootkits, de malwares et de scénarios légitimes afin d'évaluer plus largement la capacité de généralisation du détecteur et de réduire la redondance observée dans le domaine sain.

Le framework pourrait également être étendu à d'autres systèmes d'exploitation ainsi qu'à des versions plus récentes de Windows, notamment **Windows 11**, afin d'évaluer la robustesse des caractéristiques retenues face à l'évolution des mécanismes de protection du système.

L'architecture modulaire du framework ouvre également la voie à un déploiement à plus grande échelle au sein d'infrastructures **OpenStack**, permettant la supervision simultanée de plusieurs machines virtuelles et l'automatisation des analyses.

L'utilisation de **LibVMI** constitue une perspective particulièrement intéressante. En permettant une introspection mémoire directement depuis l'hyperviseur, sans nécessiter la génération de dumps mémoire complets, cette approche réduirait considérablement les besoins en stockage ainsi que le temps consacré à la capture, à l'analyse forensique et à l'extraction des caractéristiques. Elle améliorerait également la réactivité du framework face aux activités malveillantes de courte durée pouvant échapper à l'approche actuelle fondée sur des captures périodiques.

Enfin, une approche semi-automatique de découverte de caractéristiques pourrait assister l'analyste dans l'identification de nouveaux indicateurs de compromission à partir des artefacts mémoire, tout en conservant une validation humaine avant leur intégration au framework.

### Conclusion

Ce chapitre a validé le framework dans sa configuration finale, démontrant sa capacité à détecter des compromissions en mémoire. Les résultats confirment la pertinence des caractéristiques retenues ainsi que l'intérêt de l'approche implémentée associant l'apprentissage automatique à l'analyse de caractéristiques complémentaires.

## CONCLUSION GÉNÉRALE

Dans ce travail, nous nous sommes intéressés à la détection des rootkits et des malwares dans les environnements virtualisés au moyen de l'introspection mémoire des machines virtuelles. L'objectif était de concevoir un framework capable d'identifier des compromissions à partir des artefacts présents en mémoire, sans dépendre des mécanismes de surveillance internes susceptibles d'être compromis. Pour répondre à cette problématique, nous avons étudié les concepts de virtualisation, d'introspection mémoire et les principales approches de détection existantes afin de définir les exigences du framework. Nous avons ensuite mis en place un environnement expérimental basé sur OpenStack avec DevStack et développé un framework reposant sur l'acquisition de captures mémoire, leur analyse avec Volatility 3 et l'exploitation des caractéristiques extraites par un moteur de détection fondé sur l'apprentissage automatique non supervisé. Les expérimentations montrent que les performances du détecteur reposent à la fois sur la pertinence des caractéristiques extraites et sur la capacité de l'apprentissage automatique non supervisé à généraliser la détection à des variantes de rootkits non observées lors de l'entraînement. L'identification de caractéristiques robustes adaptées aux versions récentes de Windows, associée à cette capacité de généralisation, constitue l'une des principales contributions de ce travail. Ce travail ouvre plusieurs perspectives. Sur le plan expérimental, l'intégration d'un plus grand nombre de familles de rootkits et de malwares, ainsi que l'extension à d'autres systèmes d'exploitation dont Windows 11, permettrait de renforcer la validation de la capacité de généralisation du framework. Sur le plan technique, l'adoption de LibVMI en remplacement des captures mémoire périodiques réduirait sensiblement les temps d'acquisition et d'analyse, tandis que l'architecture modulaire du framework ouvre la voie à un déploiement à plus grande échelle au sein d'infrastructures OpenStack, avec supervision simultanée de plusieurs machines virtuelles. Enfin, une démarche semi-automatique de découverte de caractéristiques pourrait, à terme, assister l'analyste dans l'identification de nouveaux indicateurs de compromission.

# RÉFÉRENCES

- [1] Popek, G. J., & Goldberg, R. P. (1974). *Formal Requirements for Virtualizable Third Generation Architectures*. Communications of the ACM, 17(7), 412–421. <https://doi.org/10.1145/361011.361073> Consulté le 11 mars 2026.
- [2] Scarfone, K., Souppaya, M., & Hoffman, P. (2011). *Guide to Security for Full Virtualization Technologies*. NIST Special Publication 800-125. <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-125.pdf> Consulté le 11 mars 2026
- [3] Mell, P., & Grance, T. *The NIST Definition of Cloud Computing*. NIST Special Publication 800-145, National Institute of Standards and Technology, 2011. <https://csrc.nist.gov/publications/detail/sp/800-145/final> Consulté le 11 mars 2026
- [4] Jansen, W., & Grance, T. *Guidelines on Security and Privacy in Public Cloud Computing*. NIST SP 800-144, 2011. <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-144.pdf> Consulté le 15 mars 2026
- [5] J. Ferdous, R. Islam, A. Mahboubi et M. Z. Islam, « A Survey on ML Techniques for Multi-Platform Malware Detection: Securing PC, Mobile Devices, IoT, and Cloud Environments », *Sensors*, vol. 25, no. 4, art. 1153, 2025. <https://doi.org/10.3390/s25041153> Consulté le 21 mars 2026
- [6] Idika, N., & Mathur, A. P. (2007). A Survey of Malware Detection Techniques. Purdue University, Department of Computer Science, Technical Report. [A System for Experimentation in Replicated Copy Control](#) Consulté le 21 mars 2026
- [7] **Nadim, M., Lee, W., & Akopian, D. (2023).** *Kernel-level Rootkit Detection, Prevention and Behavior Profiling: A Taxonomy and Survey*. arXiv:2304.00473. <https://arxiv.org/abs/2304.00473> Consulté le 21 mars 2026
- [8] <https://www.kaspersky.com/about/press-releases/the-number-of-the-year-kaspersky-detected-half-a-million-malicious-files-daily-in-2025> Consulté le 21 mars 2026

- [9] A. Gaur, P. Mishra, P. Vinod, A. Singh, V. Varadharajan, U. Tupakula et M. Conti, « vDefender: An explainable and introspection-based approach for identifying emerging malware behaviour at hypervisor-layer in virtualization environment », *Computers and Electrical Engineering*, vol. 120, Part B, art. 109742, 2024 <https://doi.org/10.1016/j.compeleceng.2024.109742> Consulté le 05 Avril 2026
- [10] T. Zhang et R. B. Lee, « CloudMonatt: An Architecture for Security Health Monitoring and Attestation of Virtual Machines in Cloud Computing » [CloudMonatt](#) Consulté le 05 Avril 2026
- [11] X. Wang, J. Zhang, A. Zhang et J. Ren, « TKRD: Trusted kernel rootkit detection for cybersecurity of VMs based on machine learning and memory forensic analysis », *Mathematical Biosciences and Engineering*, vol. 16, no. 4, pp. 2650–2667, 2019 <https://www.aimspress.com/article/id/3535> Consulté le 21 Avril 2026
- [12] <https://xenbits.xen.org/docs/latest/index.html> Consulté le 15 Mai 2026
- [13] <https://www.vinchin.com/fr/vm-tips/quest-ce-que-vmware-esxi.html> Consulté le 15 Mai 2026
- [14] <https://github.com/KVM-VMI/kvm-vmi> Consulté le 15 Mai 2026
- [15] <https://docs.openstack.org/2025.1> Consulté le 15 Mai 2026
- [16] <https://github.com/volatilityfoundation/volatility3> Consulté le 20 Mai 2026
- [17] Google, *Rekall Memory Forensic Framework*. Dépôt GitHub archivé. <https://github.com/google/rekall> — Consulté le 20 mai 2026.
- [18] <https://libvmi.com/> Consulté le 20 Mai 2026
- [19] D. Challa, « Comprehensive Review of One-Class Classification Approaches for Anomaly Detection », *International Journal of Computer Applications*, vol. 186, no. 45, pp. 69–74, 2024. <https://doi.org/10.5120/ijca2024924134> Consulté le 20 Mai 2026
- [20] <https://github.com/Idov31/Nidhogg> Consulté le 01 Juin 2026
- [21] <https://github.com/redteamfortress/PhantomKiller> Consulté le 01 Juin 2026
- [22] <https://github.com/xM0kht4r/CVE-2025-7771> Consulté le 01 Juin 2026



# ANNEXE

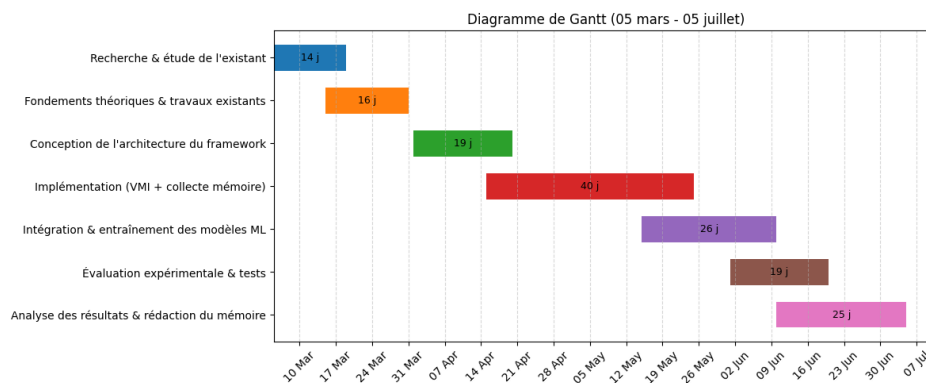


Figure A.1. Diagramme de Gantt

```

Directory: C:\Users\framework\ReflectiveDLLInjection\bin

Mode                LastWriteTime         Length Name
----                -
-a----             7/2/2026   2:58 AM           48640 inject.arm.exe
-a----             7/2/2026   2:58 AM           75264 inject.x64.exe
-a----             7/2/2026   2:58 AM           43520 reflective_dll.arm.dll
-a----             7/2/2026   2:58 AM           58880 reflective_dll.dll
-a----             7/2/2026   2:58 AM           66560 reflective_dll.x64.dll

PS C:\Users\framework\ReflectiveDLLInjection\bin> notepad.exe
PS C:\Users\framework\ReflectiveDLLInjection\bin> Get-process -Name notepad

Handles  NPM(K)    PM(K)      WS(K)      CPU(s)    Id  SI ProcessName
-----
      197      11       2248       10672        0.05   712  0  notepad

PS C:\Users\framework\ReflectiveDLLInjection\bin> .\inject.x64.exe 712
[+] Injected the 'reflective_dll.x64.dll' DLL into process 712.

```

Figure A.2. Scénario de malware en espace utilisateur par injection de la DLL reflective\_dll.x64.dll

|   |                            |          |    |    |            |      |      |                                                                             |
|---|----------------------------|----------|----|----|------------|------|------|-----------------------------------------------------------------------------|
| 9 | 2026-06-30T23:52:26.149529 | INFECTED | 99 | 99 | Userland   | 0    | 100  | /home/line/cloudguard/windows_pipeline/ml_module_windows/realtime_logs_glob |
| 8 | 2026-06-30T23:49:11.410023 | INFECTED | 99 | 99 | Userland   | 31.4 | 68.6 | /home/line/cloudguard/windows_pipeline/ml_module_windows/realtime_logs_glob |
| 7 | 2026-06-30T23:45:31.008020 | INFECTED | 99 | 99 | Userland   | 31.4 | 68.6 | /home/line/cloudguard/windows_pipeline/ml_module_windows/realtime_logs_glob |
| 6 | 2026-06-30T23:42:30.998119 | INFECTED | 99 | 99 | Noyau      | 55.6 | 44.4 | /home/line/cloudguard/windows_pipeline/ml_module_windows/realtime_logs_glob |
| 5 | 2026-06-30T23:39:30.988759 | SAIN     | 52 | 48 | Userland   | 0    | 100  | /home/line/cloudguard/windows_pipeline/ml_module_windows/realtime_logs_glob |
| 4 | 2026-06-30T23:36:30.975248 | INFECTED | 99 | 99 | Noyau      | 57.9 | 42.1 | /home/line/cloudguard/windows_pipeline/ml_module_windows/realtime_logs_glob |
| 3 | 2026-06-30T23:33:09.962723 | INFECTED | 99 | 99 | Équilibrée | 50   | 50   | /home/line/cloudguard/windows_pipeline/ml_module_windows/realtime_logs_glob |
| 2 | 2026-06-30T23:29:43.738125 | INFECTED | 99 | 99 | Userland   | 45.5 | 54.5 | /home/line/cloudguard/windows_pipeline/ml_module_windows/realtime_logs_glob |
| 1 | 2026-06-30T23:26:18.172293 | SAIN     | 97 | 3  | Userland   | 0    | 100  | /home/line/cloudguard/windows_pipeline/ml_module_windows/realtime_logs_glob |

Figure A.3. Pile d'expérimentations du détecteur



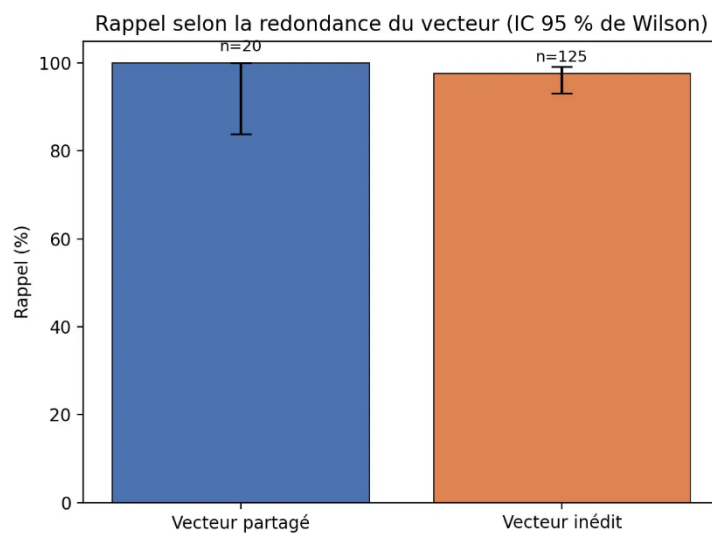


Figure A.4. Résultats du test exact de Fisher sur la redondance du vecteur

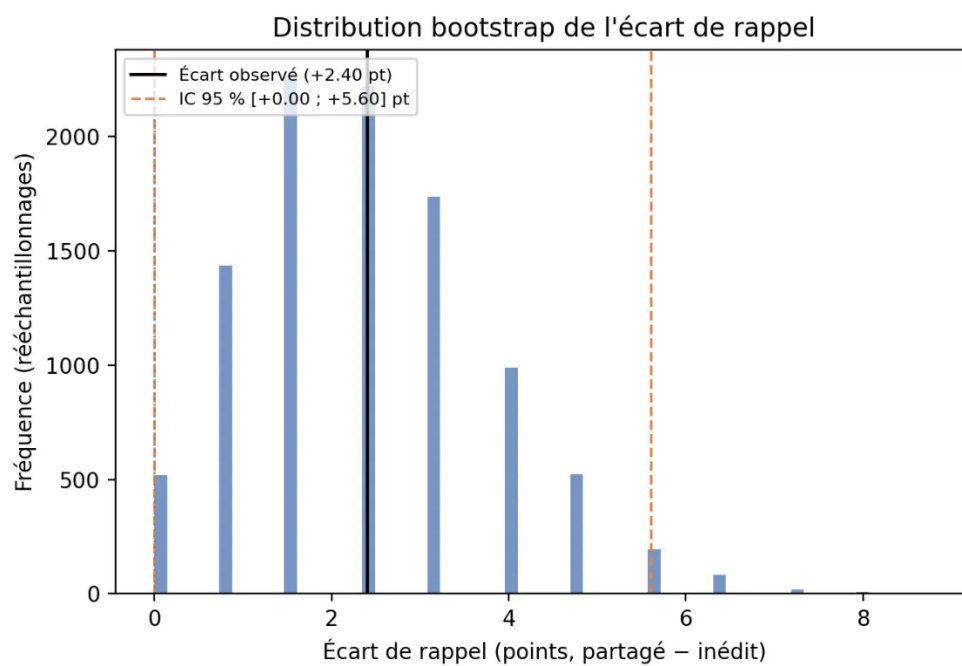


Figure A.5. Résultats du test bootstrap sur l'écart de rappel